

内容提要

希赛

- 软件架构的概念（★★★）
- 基于架构的软件开发（★★★★）
- 软件架构风格（★★★★★）
- 特定领域软件架构（★★★）
- 软件质量属性（★★★★★）
- 软件架构评估（★★★★）
- 软件产品线（★★★）
- 构件与中间件技术（★★★）
- 层次型软件架构风格（★★★★）
- 面向服务的软件架构风格（★★★★）
- 云原生架构风格（★★★★）
- Web架构综合考查（★★★★★）



软件架构的概念

希赛



架构的本质

1、软件架构为软件系统提供了一个**结构、行为和属性的高级抽象**。

2、软件架构风格是特定应用领域的**惯用模式，**架构定义**一个词汇表和一组约束。**

架构的作用

1、软件架构是**项目干系人进行交流的手段**。

2、软件架构是**可传递和可复用的模型**，通过研究软件架构可能预测软件的质量。

3、软件架构使推理和控制的更改更加简单，**有助于循序渐进的原型设计**，可以作为培训的基础。

软件架构设计是【降低成本】、【改进质量】、【按时和按需交付产品】的关键因素。



软件架构的概念

软件架构 = 软件体系结构



架构设计就是需求分配，即将满足需求的职责分配到组件上。



软件架构的概念

希赛

以下叙述，（ ）不是软件架构的主要作用。

A 在设计变更相对容易的阶段，考虑系统结构的可选方案

B 便于技术人员与非技术人员就软件设计进行交互

C 展现软件的结构、属性与内部交互关系

D 表达系统是否满足用户的功能性需求



软件架构设计与生命周期

希赛

需求分析阶段

模型转换关注两个问题：如何根据需求模型构建SA模型 + 模型转换可追溯。

设计阶段

SA研究关注得最早和最多的阶段。ADL、4+1视图。

实现阶段

构件组装阶段

在较高层次上实现系统，高效。

部署阶段

SA为部署提供高层视图指导。

后开发阶段

动态软件体系结构（内部执行和外部请求导致变化）、体系结构恢复与重建。



架构描述语言ADL

希赛

ADL是这样一种形式化语言，它在底层语义模型的支持下，为软件系统的概念体系结构建模提供了具体语法和概念框架。

C2SADL【基于组件和消息的软件架构描述语言】

Wright【分布、并发类型的架构描述语言】

ACME【架构互换语言】

UniCon【基于组件和连接的架构描述语言】

Rapide【基于事件的架构描述语言】

其他【Darwin、MetaH、Aesop、Weaves、SADL、xADL】

```
USES p1 PROTOCOL Unix-pipe
USES sorter INTERFACE Sort-filter
CONNECT sorter.output TO p1.source
USES p2 PROTOCOL Unix-pipe
USES printer INTERFACE Print-filter
CONNECT sorter.input TO p2.sink
```

ADL的三个基本元素

构件：计算或数据存储单元。

连接件：用于构件之间交互建模的体系结构构造块及其支配这些交互的规则。

架构配置：描述体系结构的构件与连接件的连接图。



架构描述语言ADL

希赛

架构描述语言（Architecture Description Language, ADL）是一种为明确说明软件系统的概念架构和对这些概念架构建模提供功能的语言。ADL主要包括以下组成部分：组件、组件接口、（ ）和架构配置。

A 架构风格
B 架构实现
C 连接件
D 组件约束

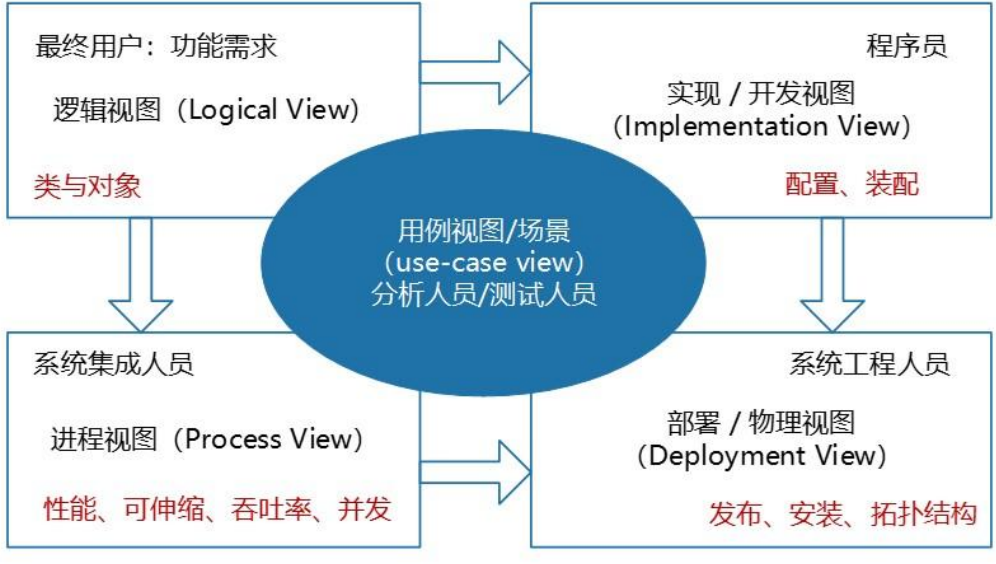


“4+1”视图

希赛

视角与视图

从不同的视角来检查，所以会有不同的视图。





架构的“4+1”视图

希赛

1995年Kruchten提出了著名的“4+1”视图，用来描述软件系统的架构。在“4+1”视图中，（）用来描述设计的对象模型和对象之间的关系；（）描述了软件模块的组织与管理；（）描述设计的并发和同步特征。

A 逻辑视图　　B 用例视图　　C 过程视图　　D 开发视图

A 逻辑视图　　B 用例视图　　C 过程视图　　D 开发视图

A 逻辑视图　　B 用例视图　　C 过程视图　　D 开发视图



基于架构的软件开发方法－概念

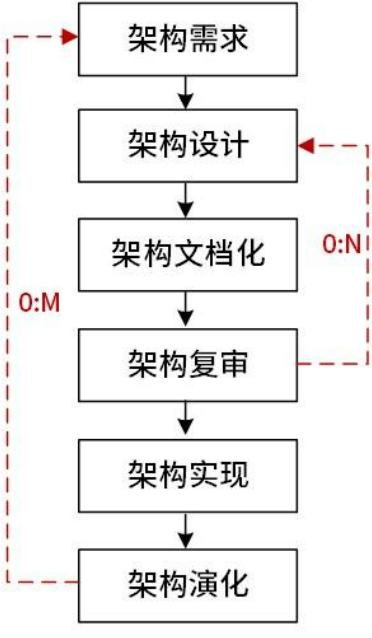
希赛

- ✓ ABSD方法是架构驱动，即强调由业务【商业】、质量和功能需求的组合驱动架构设计。
- ✓ ABSD方法有三个基础。第一个基础是功能的分解。在功能分解中，ABSD方法使用已有的基于模块的内聚和耦合技术；第二个基础是通过选择架构风格来实现质量和业务需求；第三个基础是软件模板的使用。
- ✓ 视角与视图：从不同的视角来检查，所以会有不同的视图。
- ✓ 用例用来**捕获功能需求**、特定场景【刺激、环境、响应】用来**捕获质量需求**。



基于架构的软件开发方法 – 开发过程

希赛



ABSD能很好的【支持软件重用】

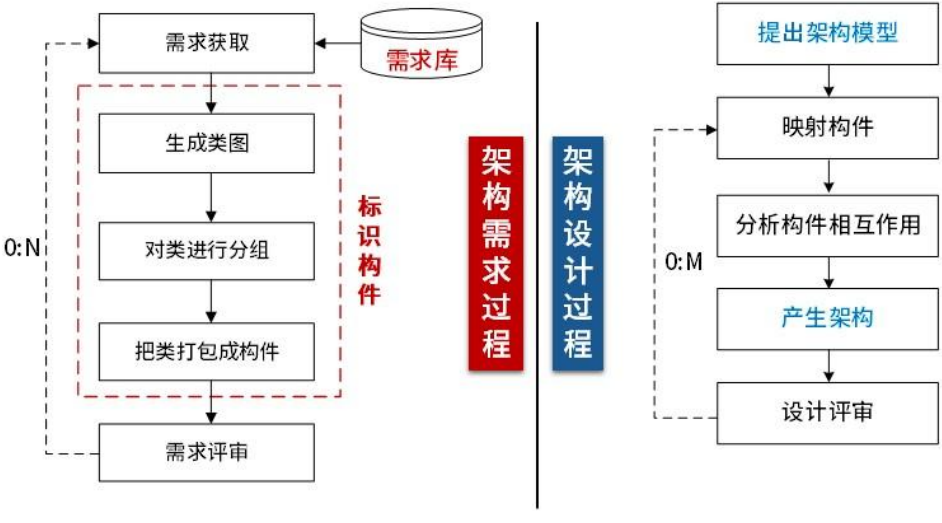
ABSD方法是一个自顶向下，递归细化的方法。

软件系统的体系结构通过该方法得到细化，直到能产生软件构件和类。



基于架构的软件开发方法 – 开发过程

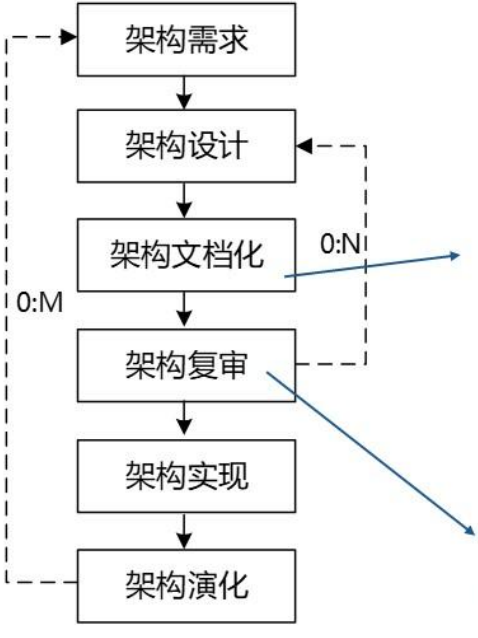
希赛





基于架构的软件开发方法 – 开发过程

希赛



架构文档化过程的主要输出结果是**架构规格说明**和**测试架构需求的质量设计说明书**这两个文档。

文档的**完整性和质量**是软件架构成功的关键因素。

关于文档的三大注意事项：

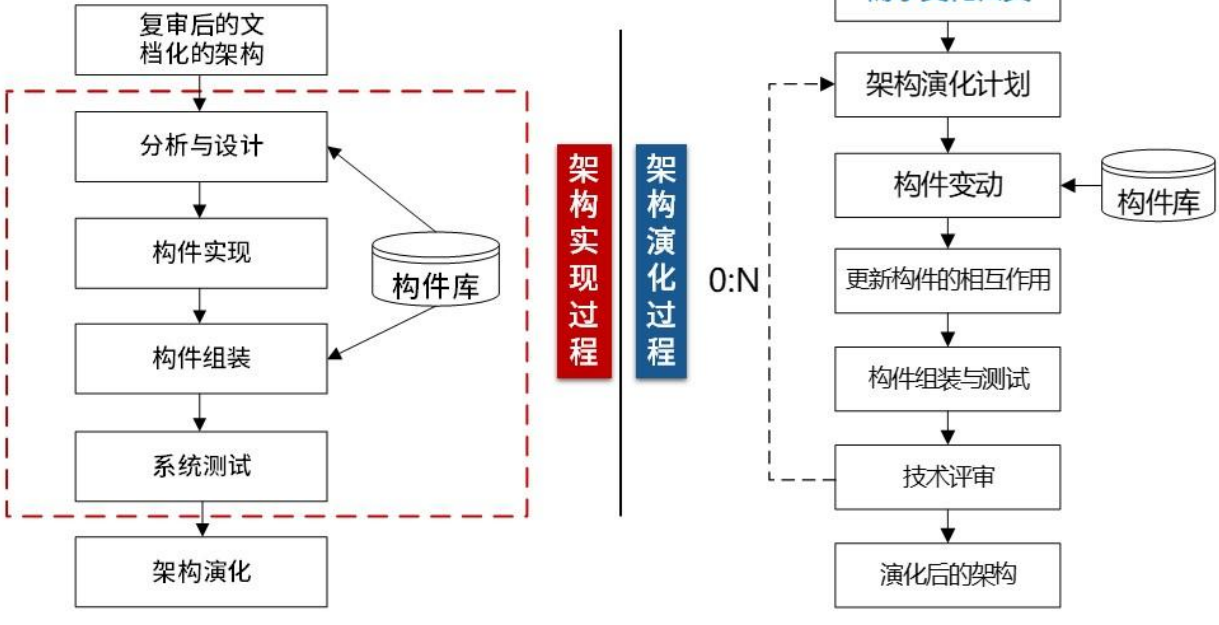
- ✓ 文档要从使用者的角度进行编写
- ✓ 必须分发给所有与系统有关的开发人员
- ✓ 且必须保证开发者手上的文档是最新的

架构复审【架构评估】的目的是**标识潜在的风险**，及早发现架构设计中的缺陷和错误。



基于架构的软件开发方法 – 开发过程

希赛





基于架构的软件开发方法 – 练习题

希赛

在基于体系结构的软件设计方法中，采用（ ）来描述软件架构，采用（ ）来描述功能需求，采用（ ）来描述质量需求。

A 类图和序列图　　B 视角与视图
C 构件和类图　　D 构件与功能

A 类图　　B 视角
C 用例　　D 质量场景

A 连接件　　B 用例
C 质量场景　　D 质量属性



基于架构的软件开发方法 – 练习题

希赛

ABSDM（Architecture-Based Software Design Model）把整个基于体系结构的软件过程划分为体系结构需求、体系结构设计、体系结构文档化、（ ）、（ ）和体系结构演化等 6个子过程。其中，（ ）过程的主要输出结果是体系结构规格说明和测试体系结构需求的质量设计说明书。

A 体系结构复审　　B 体系结构测试
C 体系结构变更　　D 体系结构管理

A 体系结构实现　　B 体系结构测试
C 体系结构建模　　D 体系结构管理

A 体系结构设计　　B 体系结构需求
C 体系结构文档化　　D 体系结构测试



软件架构风格

希赛

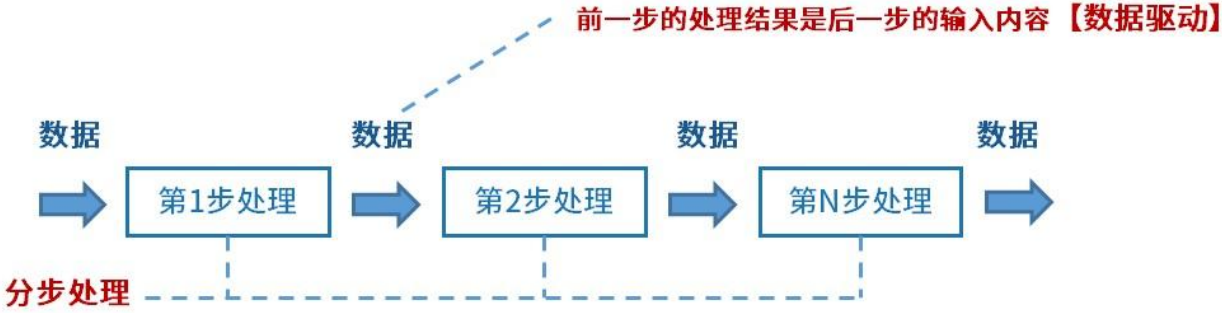
- 架构风格定义了用于描述系统的术语表和一组指导构建系统的规则。

五大架构风格	子风格
数据流风格 【Data Flow】	批处理【Batch Sequential】、管道-过滤器【Pipes and Filters】
调用/返回风格 【Call/Return】	主程序/子程序【Main Program and Subroutine】 面向对象【Object-oriented】、分层架构【Layered System】
独立构件风格 【Independent Components】	进程通信【Communicating Processes】 事件驱动系统（隐式调用）【Event system】
虚拟机风格 【Virtual Machine】	解释器【interpreter】、规则系统【Rule-based System】
以数据为中心 【Data-centered】	数据库系统【Database System】、黑板系统【Blackboard System】 超文本系统【Hypertext System】



软件架构风格 – 数据流风格

希赛

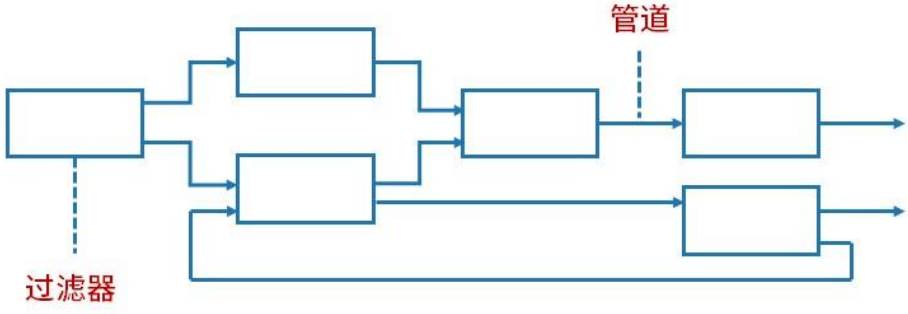


优点	缺点	典型实例
1、松耦合【高内聚-低耦合】； 2、良好的重用性/可维护性； 3、可扩展性【标准接口适配】； 4、良好的隐蔽性； 5、支持并行。	1、交互性较差； 2、复杂性较高； 3、性能较差（每个过滤器都需要解析与合成数据）。	传统编译器 网络报文处理



软件架构风格 – 数据流风格

希赛



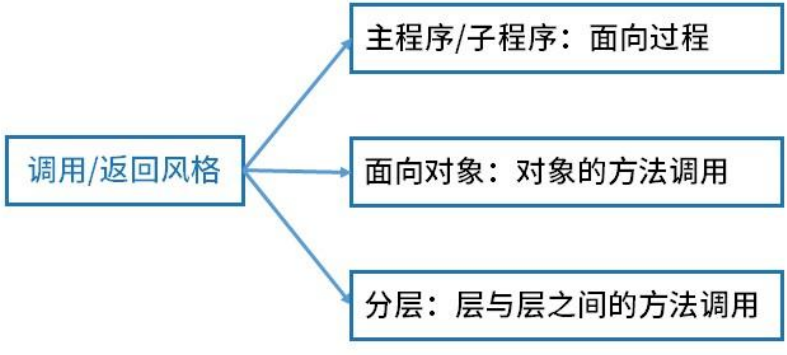
- 数据流风格
  - 批处理序列：大量整体数据、无需用户交互
  - 管道-过滤器：流式数据、弱用户交互



软件架构风格 – 调用/返回风格

希赛



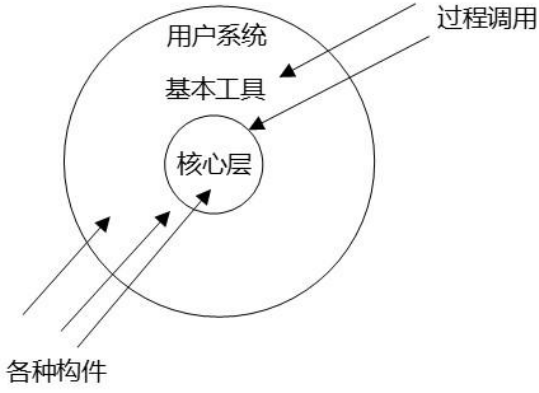




软件架构风格 – 调用/返回风格

希赛

分层架构风格

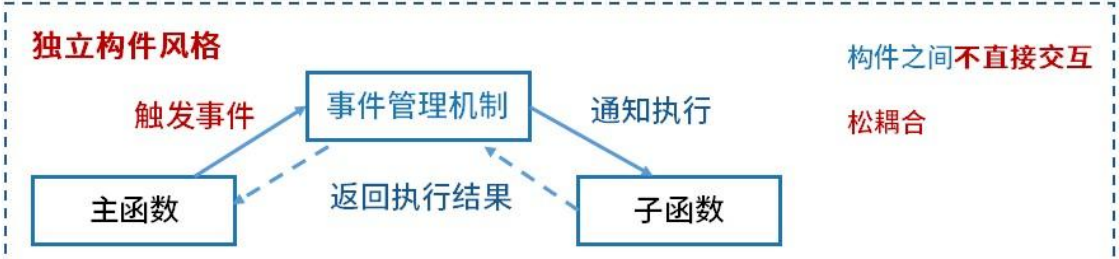


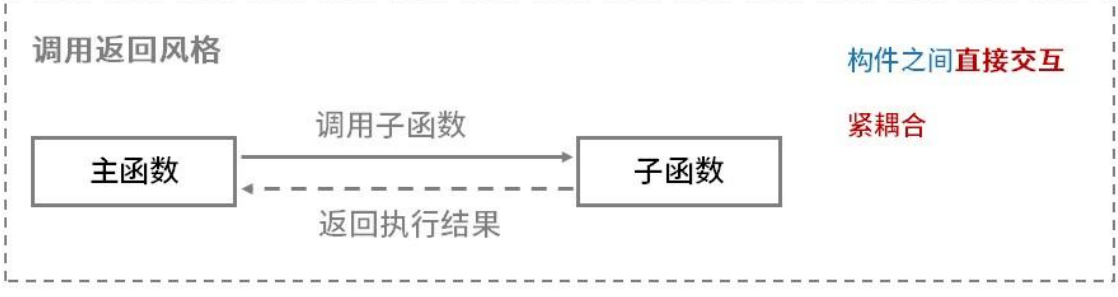
优点	缺点	特点
1、良好的重用性，只要接口不变可用在其它处； 2、可维护性好； 3、可扩展性好，支持递增设计。	1、并不是每个系统都方便分层； 2、很难找到一个合适的、正确的层次抽象方法； 3、不同层次之间耦合度高的系统很难实现。	各个层次的组件形成不同功能级别的虚拟机； 多层相互协同工作，而且实现透明。



软件架构风格 – 独立构件风格

希赛







软件架构风格 – 独立构件风格

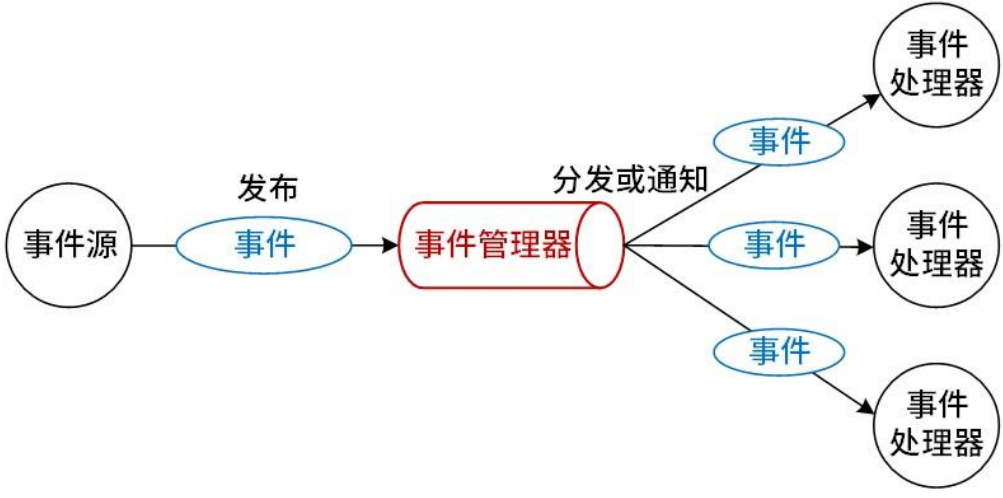
希赛

优点	缺点	特点
1、松耦合。 2、良好的重用性/可修改性/可扩展性。	1、构件放弃了对系统计算的控制。一个构件触发一个事件时，不能确定其他构件是否会响应它。而且即使它知道事件注册了哪些构件的过程，它也不能保证这些过程被调用的顺序。 2、数据交换的问题。 3、既然过程的语义必须依赖于被触发事件的上下文约束，关于正确性的推理就存在问题。	系统由若干子系统构成且成为一个整体；系统有统一的目标；子系统有主从之分；每一子系统有自己的事件收集和处理机制



软件架构风格 – 独立构件风格

希赛

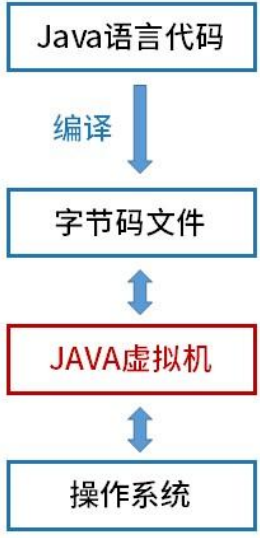




软件架构风格 – 虚拟机风格

希赛







软件架构风格 – 虚拟机风格

希赛





软件架构风格 – 虚拟机风格

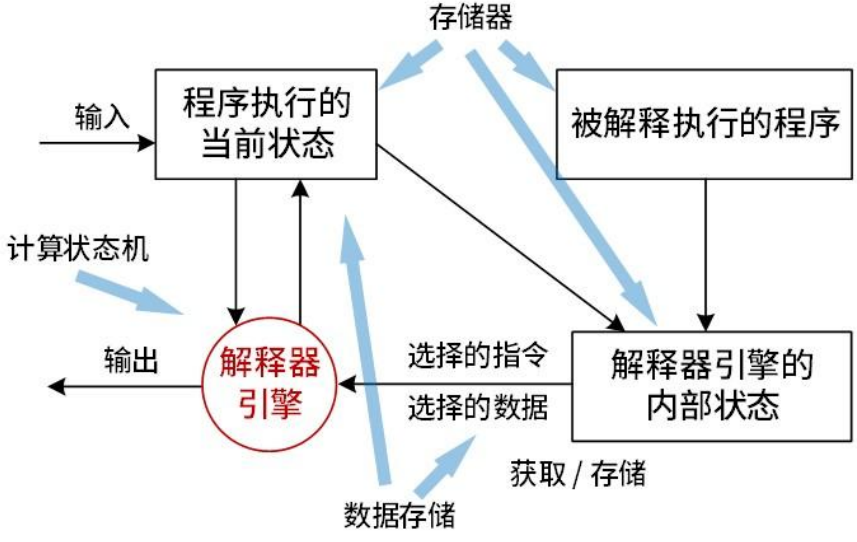
希赛

子分类	优点	缺点	特点	适合领域
解释器	可以灵活应对自定义场景	复杂度较高		适用于需要“自定义规则”的场合
规则为中心			在解释器的基础上增加经验规则	适用于专家系统



软件架构风格 – 虚拟机风格

希赛



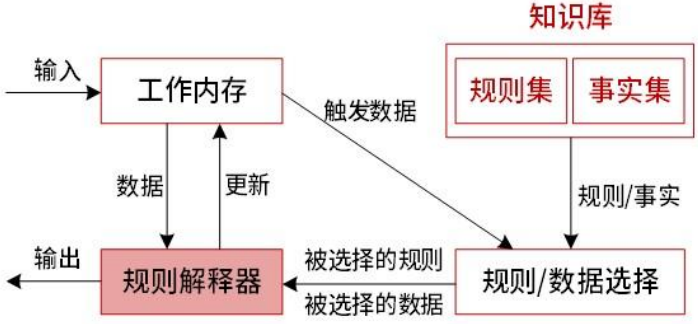
解释器体系结构风格



软件架构风格 – 虚拟机风格

希赛

基于规则的系统构成：规则集、规则解释器、规则/数据选择及工作内存，一般用在人工智能领域和DSS（决策支持系统）中。

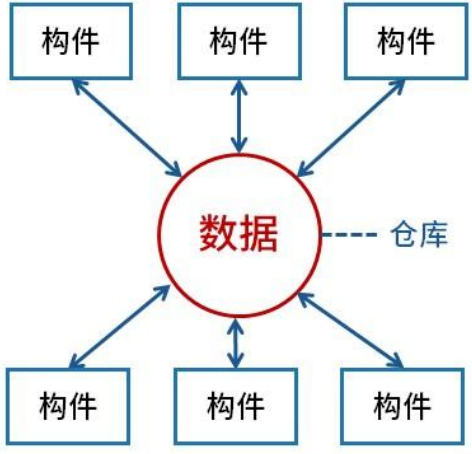


规则系统体系结构风格



软件架构风格 – 以数据为中心

希赛



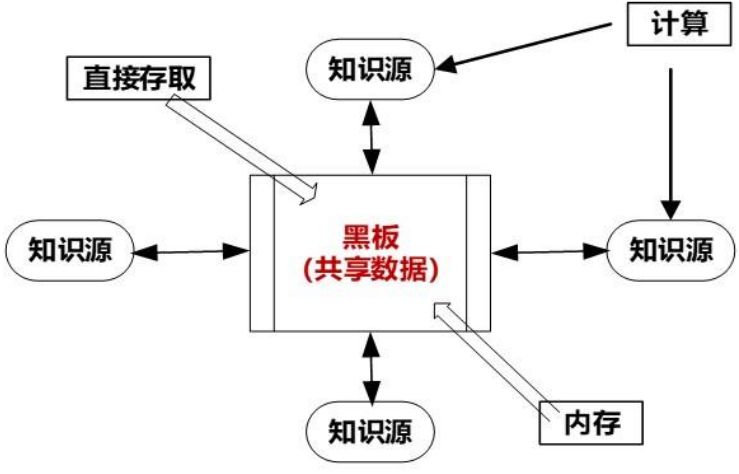
以数据为中心
【仓库风格】

- 数据库系统
- 黑板系统：语音识别、知识推理
- 超文本系统



软件架构风格 – 以数据为中心

希赛





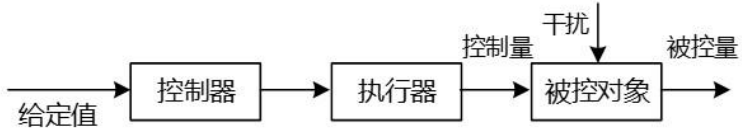
软件架构风格 – 以数据为中心

希赛

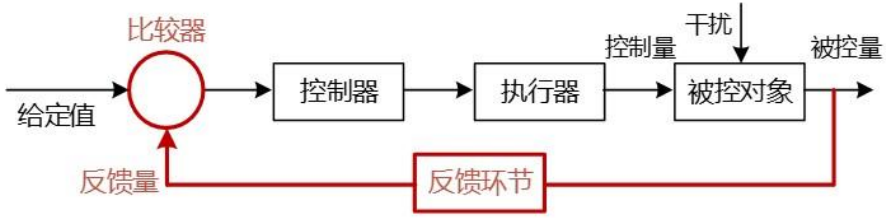
架构风格	子分类	优点	缺点	特点	典型实例
以数据为中心	数据库系统			以数据为中心	
	黑板系统	可更改性和可维护性；可重用的知识源；容错性和健壮性	测试困难；不能保证有好的解决方案；难以建立好的控制策略；低效；开发困难；缺少并行机制	在以数据为中心的基础上，使用中心数据触发业务逻辑部件	语音识别 模式识别 图像处理 知识推理



软件架构风格 – 闭环控制架构（过程控制）



开环控制系统



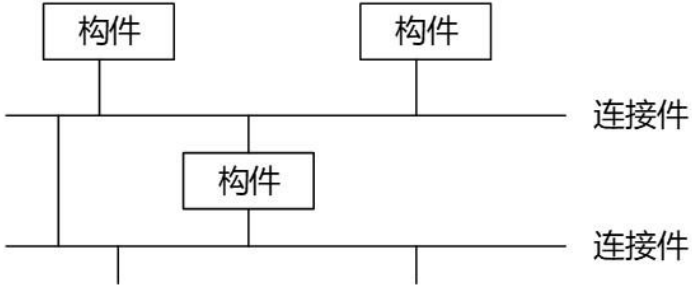
闭环控制系统

- ✓ 适合于嵌入式系统，**用于解决简单闭环控制问题**。
- ✓ **经典应用：空调温控，定速巡航。**



软件架构风格 – C2风格

希赛



C2架构的基本规则：

- ✓ **构件和连接件都有一个顶部和一个底部。**
- ✓ 构件的顶部要连接到连接件的底部，构件的底部要连接到连接件的顶部，构件之间不允许直连。
- ✓ 一个连接件可以和任意数目的其他构件和连接件连接。
- ✓ 当两个连接件进行直接连接时，必须由其中一个的底部到另一个的顶部。



软件架构风格 – 架构风格判断

希赛

- Java程序可以做到“一次编写，到处运行”，从架构风格上看符合（ ）风格的特点。
- 在网络通信中，进行包的解析，一般先进行包头的分离，然后进行报文解析及后续处理，根据这一特点，选用（ ）风格最合适。
- 某公司欲开发一个基于图形用户界面的集成调试器。该调试器的编辑器和变量监视器可以设置调试断点。当调试器在断点处暂停运行时，编辑程序可以自动卷屏到断点，变量监视器刷新变量数值。针对这样的功能描述，采用（ ）的架构风格最为合适。
- 某游戏公司欲开发一个大型多人即时战略游戏，游戏设计的目标之一是能够支持玩家自行创建战役地图，定义游戏对象的行为和对象之间的关系。针对该目标，公司应该采用（ ）架构风格最为合适。（四选一：管道-过滤器、隐式调用、主程序-子程序、解释器）
- 某公司承接了一个开发家用空调自动调温器的任务，调温器测量外部空气温度，根据设定的期望温度控制空调的开关。根据该需求，公司应采用（ ）架构风格最为合适。（四选一：解释器、过程控制、分层、管道-过滤器）



软件架构风格 – 架构风格判断

希赛

- 某公司欲开发一个语音识别系统，语音识别的主要过程包括分割原始语音信号、识别音素、产生候选词、判定语法片段、提供语义解释等。每个过程都需要进行基于先验知识的条件判断并进行相应的识别动作。针对该系统的特点，采用（ ）架构风格最为合适。（四选一：解释器、面向对象、黑板、隐式调用）
- 某公司欲开发一个漫步者机器人，用来完成火星探测任务。机器人的控制者首先定义探测任务和任务之间的时序依赖性，机器人接受任务后，需要根据自身状态和外界环境进行动态调整，最终自动完成任务。针对这些需求，该机器人应该采用（ ）架构风格最为合适。（四选一：解释器、主程序-子程序、隐式调用、管道-过滤器）
- Windows操作系统在图形用户界面处理方面采用的核心架构风格是（ ）风格。



软件架构风格－练习题

希赛

“编译器”是一种非常重要的基础软件，其核心功能是对源代码形态的单个或一组源程序依次进行预处理、词法分析、语法分析、语义分析、代码生成、代码优化等处理，最终生成目标机器的可执行代码。考虑以下与编译器相关的软件架构设计场景：传统的编译器设计中，上述处理过程都以独立功能模块的形式存在，程序源代码作为一个整体，依次在不同模块中进行传递，最终完成编译过程。针对这种设计思路，传统的编译器采用（1）架构风格比较合适。

随着编译、链接、调试、执行等开发过程的一体化趋势发展，集成开发环境（IDE）随之出现。IDE集成了编译器、连接器、调试器等多种工具，支持代码的增量修改与处理，能够实现不同工具之间的信息交互，覆盖整个软件开发生命周期。针对这种需求，IDE采用（2）架构风格比较合适。IDE强调交互式编程，用户在修改程序代码后，会同时触发语法高亮显示、语法错误提示、程序结构更新等多种功能的调用与结果呈现，针对这种需求，通常采用（3）架构风格比较合适。

某公司已经开发了一款针对某种嵌入式操作系统专用编程语言的IDE，随着一种新的嵌入式操作系统上市并迅速占领市场，公司决定对IDE进行适应性改造，支持采用现有编程语言进行编程，生成符合新操作系统要求的运行代码，并能够在现有操作系统上模拟出新操作系统的运行环境，以支持代码调试工作。针对上述要求，为了使IDE能够生成符合新操作系统要求的运行代码，采用基于（4）的架构设计策略比较合适；为了模拟新操作系统的运行环境，通常采用（5）架构风格比较合适。

（1）A．管道-过滤器	B．顺序批处理	C．过程控制	D．独立进程
（2）A．规则引擎	B．解释器	C．数据共享	D．独立构件
（3）A．隐式调用	B．显式调用	C．主程序-子程序	D．层次结构
（4）A．代理	B．适配	C．包装	D．模拟
（5）A．隐式调用	B．仓库结构	C．基于规则	D．虚拟机



软件架构风格 – MDA

希赛

Model Driven Architecture

【模型驱动架构】

– Model ?

客观事物的抽象表示

– Architecture ?

构成系统的部件、连接件及其约束的规约

– Model-Driven ?

使用模型完成软件的分析、设计、构建、部署、维护等各开发活动

– MDA起源于分离系统规约和平台实现的思想

– MDA的主要目标:

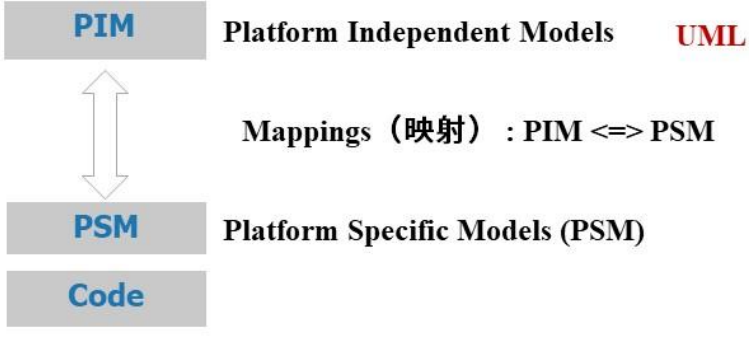
Portability（可移植性）, interoperability（互通性）,
Reusability（可重用性）



软件架构风格 – MDA

希赛

- MDA的核心模型：
  - 计算无关模型（CIM）：对某具体行业内一个项目的业务需求及其系统功能需求进行分析
  - 平台独立模型（PIM）：具有高抽象层次、独立于任何实现技术的模型。
  - 平台相关模型（PSM）：为某种特定实现技术量身定做，让你用这种技术中可用的实现构造来描述系统的模型。PIM会被变换成一个或多个PSM。
  - 代码Code：用源代码对系统的描述（规约）。每个PSM都将被变换成代码。







软件架构复用

希赛

★ 【软件复用】是一种系统化的软件开发过程，通过识别、分析、分类、获取和修改软件实体，以便在不同软件开发过程中重复使用它们。

★ 软件开发过程中重复使用相同或相似【软件元素】的过程。

复用的历史发展线路



复用的维度

水平复用：不分行业领域，通用

垂直复用：分行业领域，专用



软件架构复用

希赛

软件架构复用类型	机会复用	开发过程中，只要发现有可复用资产，就对其进行复用
	系统复用	开发之前，要进行规划，以决定哪些需要复用

【可复用的资产范围有多广？】需求、架构设计、元素、建模与分析、测试、项目规划、过程、方法和工具、人员、样本系统、缺陷消除。



获取可复用的软件资产　　构件分类+构件检索　　修改、扩展、配置、组装、集成【形成最终系统】



软件架构复用

软件重用可以分为垂直式重用和水平式重用，（　）是一种典型的水平式重用。

A 医学词汇表　　B 标准函数库　　C 电子商务标准　　D 网银支付接口

（　）是指在开发之前，就要进行规划，以决定哪些需要复用。

A 预期复用　　B 系统复用　　C 计划复用　　D 机会复用

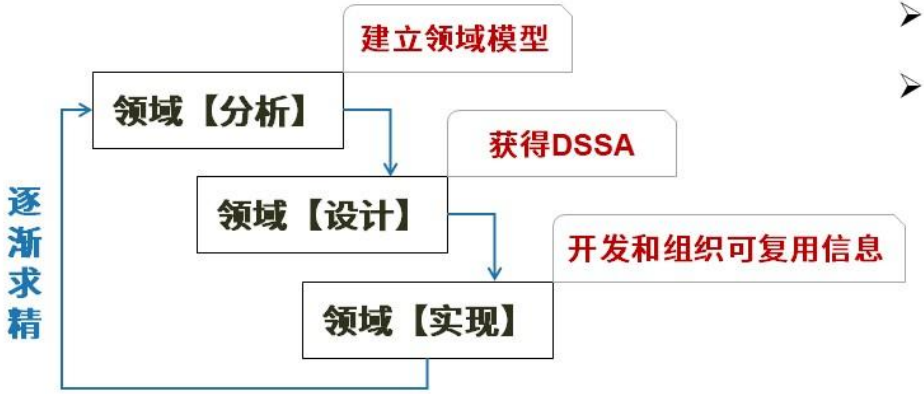


特定领域软件架构（DSSA）– 基本概念

希赛

定义：特定领域软件架构以一个特定问题领域为对象，形成由领域参考模型、参考需求、参考架构等组成的开发基础架构，支持一个特定领域中多个应用的生成。

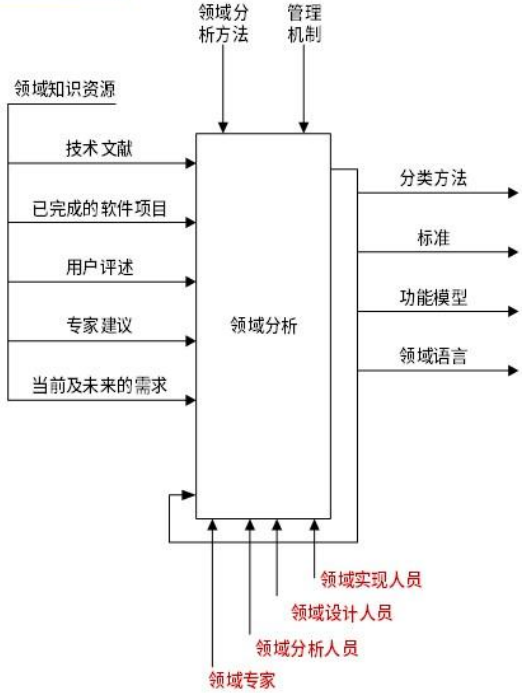
DSSA基本活动及产出物



DSSA类型

- 垂直域：相同领域，深入
- 水平域：不同领域，平移

特定领域软件架构（DSSA）– 参与人员



参与DSSA的人员

1、领域专家：有经验的用户、从事该领域中系统的需求分析、设计、实现以及项目管理的有经验的软件工程师等。

领域专家的主要任务包括提供关于领域中系统的需求规约和实现的知识。

2、领域分析人员：领域分析人员应由具有知识工程背景的有经验的系统分析员来担任。

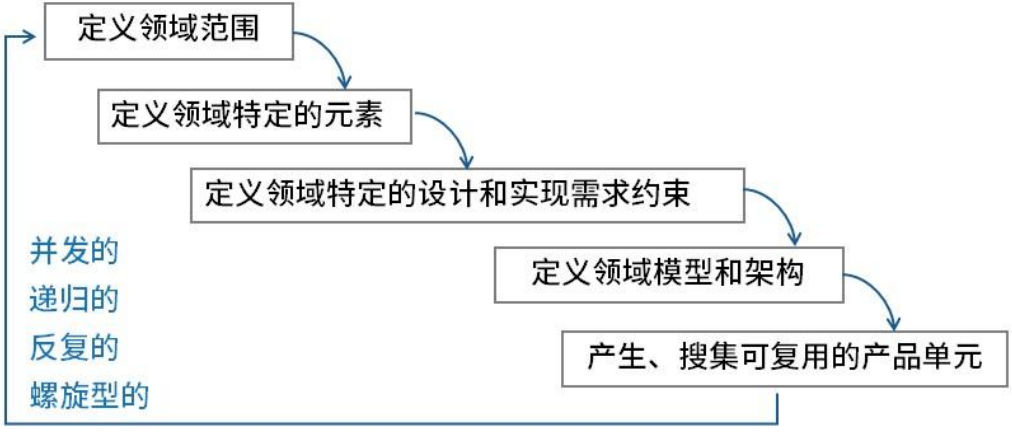
3、领域设计人员：领域设计人员应由有经验的软件设计人员来担任。

4、领域实现人员：领域实现人员应由有经验的程序设计人员来担任。



特定领域软件架构(DSSA) – 建立过程

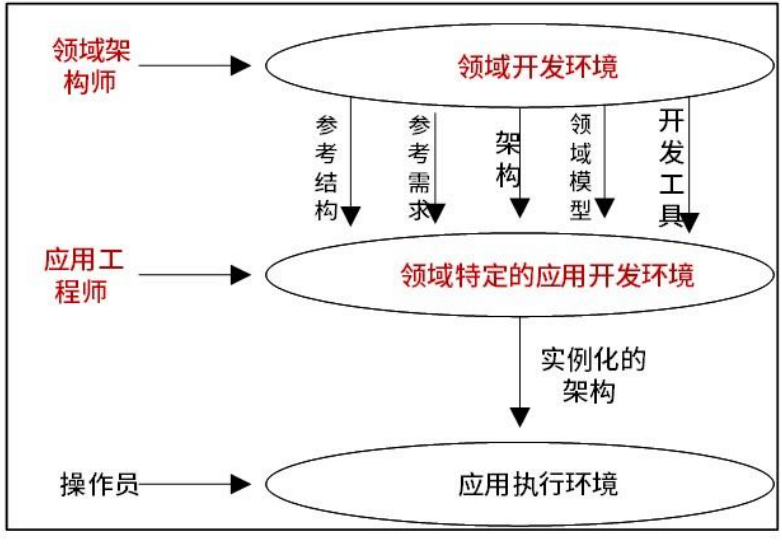
希赛





特定领域软件架构（DSSA）– 三层次模型

希赛





特定领域软件架构（DSSA）

希赛

DSSA是在一个特定应用领域中为一组应用提供组织结构参考的软件体系结构，参与DSSA的人员可以划分为4种角色，包括领域专家、领域设计人员、领域实现人员和（ ），其基本活动包括领域分析、领域设计和（ ）。

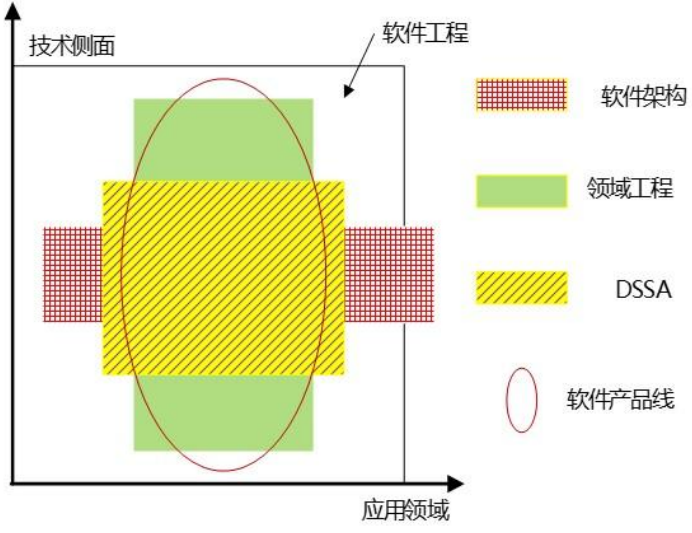
A 领域测试人员　B 领域顾问　C 领域分析师　D 领域经理

A 领域建模　B 架构设计　C 领域实现　D 领域评估



软件产品线

希赛



【核心资源库】+【产品集合】

产品构造的基础

领域工程的所有结果的集合



软件架构评估

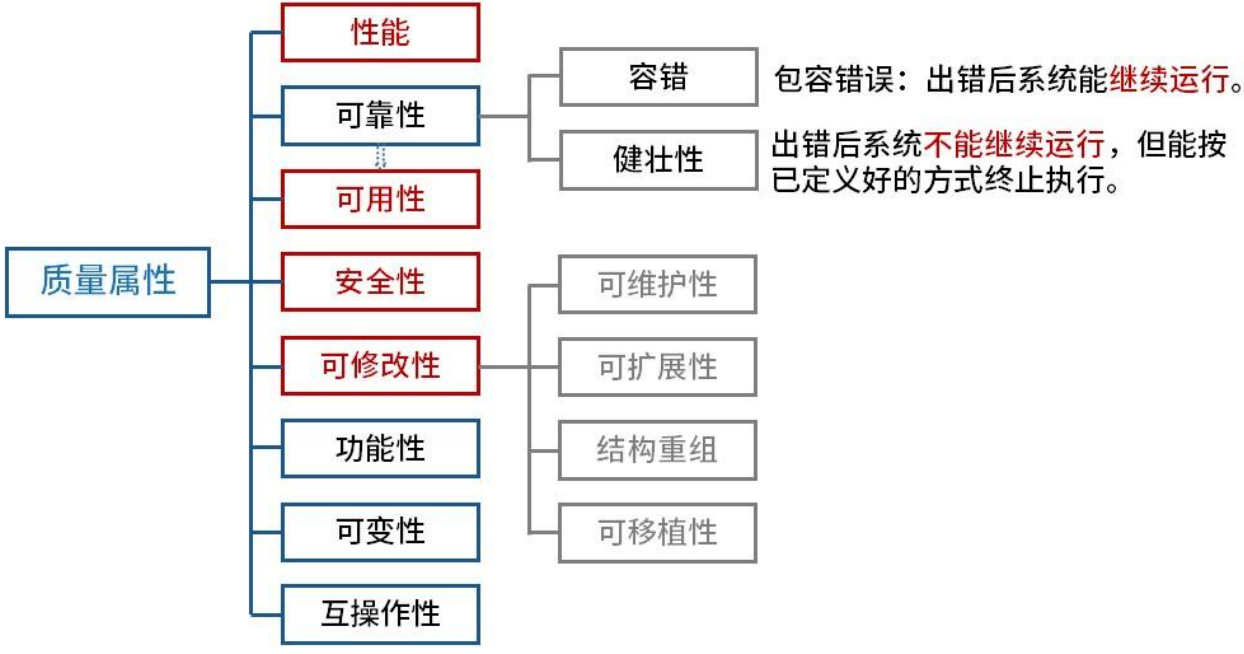
希赛

- ✓ 为什么要进行架构评估?
- ✓ 架构评估到底评什么?
- ✓ 架构评估怎么评?



软件架构评估－质量属性

希赛





软件架构评估 – 质量属性

希赛

【开发期】质量属性

（1）易理解性：指设计被开发人员理解的难易程度。

（2）可扩展性：软件因适应新需求或需求变化而增加新功能的能力，也称为灵活性。

（3）可重用性：指重用软件系统或某一部分的难易程度。

（4）可测试性：对软件测试以证明其满足需求规范的难易程度。

（5）可维护性：修改缺陷、增加功能、提高质量属性时，识别修改点并修改的难易程度。

（6）可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度

【运行期】质量属性

（1）性能：性能是指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。

（2）安全性：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。

（3）可伸缩性：用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。

（4）互操作性：指本软件系统与其他系统交换数据和相互调用服务的难易程度。

（5）可靠性：软件系统一定时间内持续无故障运行的能力。

（6）可用性：系统在一定时间内正常工作时间所占的比例。可用性会受到系统错误，恶意攻击，高负载等问题的影响。

（7）鲁棒性：软件系统在非正常情况（非法操作、软硬件故障等）下仍能够正常运行的能力，也称健壮性或容错性。



软件架构评估－质量属性

希赛

1、性能

性能（performance）是指**系统的响应能力**，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的个数。

例如：

（1）同时支持1000并发；

（2）响应时间小于1s；

（3）显示分辨率达到4K。





软件架构评估 – 质量属性

希赛

2、可用性

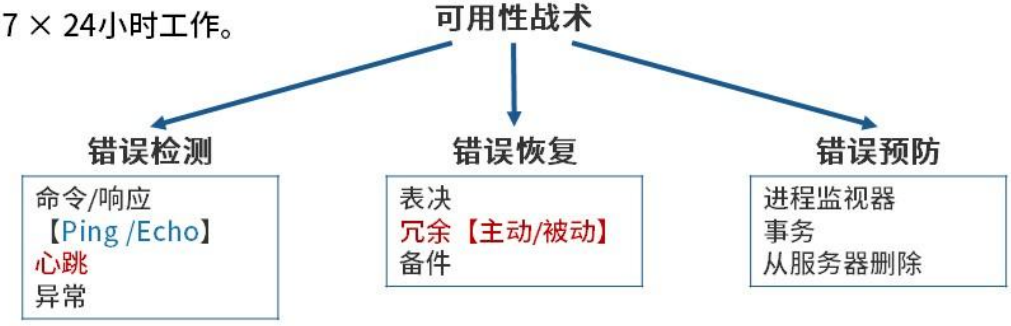
可用性（availability）是系统能够**正常运行的时间比例**。经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。

例如：

（1）主服务器故障，1分钟内切换至备用服务器；

（2）系统故障，1小时内修复；

（3）系统支持7 × 24小时工作。





软件架构评估－质量属性

希赛

3、安全性

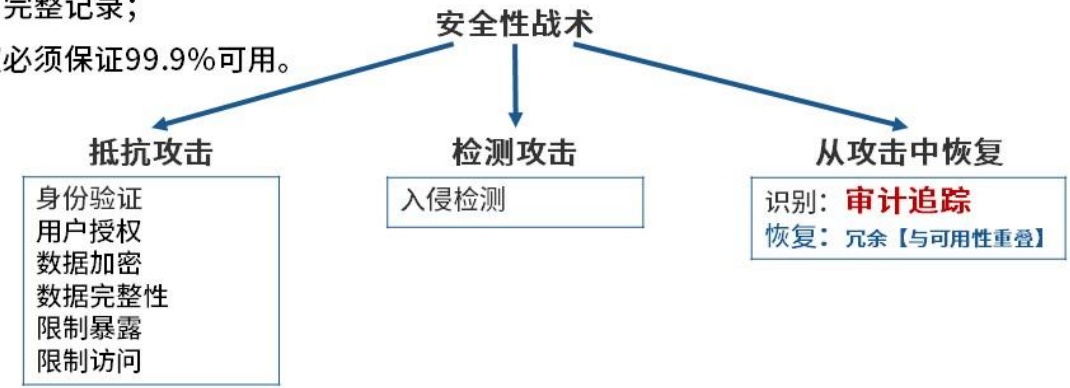
安全性（security）是指系统在**向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力**。安全性又可划分为机密性【信息不泄露给未授权的用户】、完整性【防止信息被篡改】、不可否认性【不可抵赖】及可控性【对信息的传播及内容具有控制的能力】等特性。

例如：

（1）可抵御SQL注入攻击；

（2）对计算机的操作都有完整记录；

（3）用户信息数据库授权必须保证99.9%可用。





软件架构评估－质量属性

希赛

4、可修改性

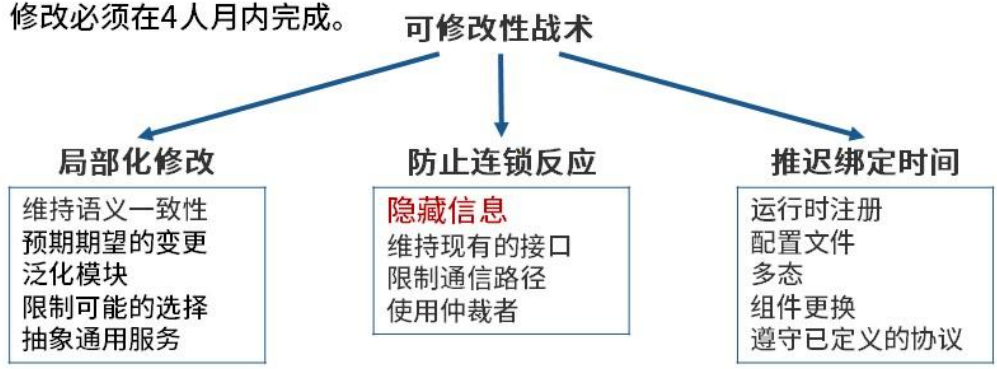
可修改性（modifiability）是指能够快速地以**较高的性能价格比对系统进行变更的能力**。通常以某些具体的变更为基准，通过考察这些变更的代价衡量可修改性。

可修改性包含4个方面：可维护性、可扩展性、结构重组、可移植性。

例如：

（1）更改系统报表模块，必须在2人周内完成；

（2）对Web界面风格进行修改，修改必须在4人月内完成。





5、易用性

易用性关注的是对用户来说完成某个期望任务的容易程度和系统所提供的用户支持的种类。

例如：

（1）界面友好；

（2）新用户学习使用系统时间不超过2小时。

6、可测试性

软件可测试性是指通过测试揭示软件缺陷的容易程度。

例如：

（1）提供远程调试接口，支持远程调试。



软件架构评估－质量属性

希赛

某服务器软件系统对可用性（Availability）、性能（Performance）和可修改性（Modification）的要求较高，（ ）设计策略能提高该系统的可用性，（ ）设计策略能够提高该系统的性能，（ ）设计策略能够提高该系统的可修改性。

A Ping/Echo	B 限制访问	C 运行时注册	D 接口-实现分离
A 分层结构	B 事务机制	C 主动冗余	D 队列调度
A 信息隐藏	B 记录/回放	C 任务模型	D 回滚



软件架构评估－质量属性

希赛

某公司欲开发一个电子交易清算系统，在架构设计阶段，公司的架构师识别出3个核心质量属性场景。其中“数据传递时延不大于1s，并提供相应的优先级管理”主要与（ ）质量属性相关，通常可采用（ ）架构策略实现该属性；“系统采用双机热备，主备机必须实时监测对方状态，以便完成系统的实时切换”主要与（ ）质量属性相关，通常可采用（ ）架构策略实现该属性；“系统应能够防止99%的黑客攻击”主要与（ ）质量属性相关，通常可采用（ ）架构策略实现该属性。

A 可用性	B 性能	C 安全性	D 可修改性
A 限制资源	B 引入并发	C 资源仲裁	D 限制访问
A 可用性	B 性能	C 安全性	D 可修改性
A 记录/回放	B 操作串行化	C 心跳	D 资源调度
A 可用性	B 性能	C 安全性	D 可修改性
A 检测攻击	B Ping /Echo	C 选举	D 权限控制



软件架构评估

希赛

敏感点是一个或多个构件（和/或构件之间的关系）的特性，它能影响系统的某个质量属性。

权衡点是影响多个质量属性的特性，是多个质量属性的敏感点。



风险点是指架构设计中潜在的、存在问题的架构决策所带来的隐患。

非风险点是指不会带来隐患，一般以“XXX要求是可以实现（或接受）的”方式表达。

例如：

（1）对交易请求处理时间的要求将影响系统的数据传输协议和处理过程的设计；

（2）假设每秒中用户交易请求的数量是10个，处理请求的时间为30毫秒，则“在1秒内完成用户的交易请求”这一要求是可以实现的；

（3）目前对系统信用卡支付业务逻辑的描述尚未达成共识，这可能导致部分业务功能模块的重复，影响系统的可修改性；

（4）更改加密的级别将对安全性和性能产生影响。



软件架构评估

希赛

	性能	安全性	可用性	易用性	可测试性	可维护性
性能		-		-	-	-
安全性	-			-	-	
可用性						
易用性	-		-		-	
可测试性	-					+
可维护性	-		+		+	



软件架构评估 – 架构评估方法

希赛

- ✓ 基于调查问卷（检查表）的方式
- ✓ 基于度量的方式
- ✓ **基于场景的方式**

评估方式	调查问卷或检查表		场景	度量
	调查问卷	检查表		
通用性	通用	特定领域	特定系统	通用或特定领域
评估者对架构的了解程度	粗略了解	无限制	中等了解	精确了解
实施阶段	早	中	中	中
客观性	主观	主观	较主观	较客观



软件架构评估－架构评估方法

希赛

【质量属性场景】是从风险承担者的角度与系统交互的简短描述，它通常作为描述质量属性的手段。

场景可从六个方面进行描述：刺激源、刺激、制品、环境、响应、响应度量。

【性能场景示例】





软件架构评估－架构评估方法

希赛

质量属性场景的六个构成部分：

【刺激源（Source）】这是某个生成该刺激的实体（人、计算机系统或者任何其他刺激器）。

【刺激（Stimulus）】该刺激是当刺激到达系统时需要考虑的条件。

【环境（Environment）】该刺激在某些条件内发生。当激励发生时，系统可能处于过载、运行或者其他情况。

【制品（Artifact）】某个制品被激励。这可能是整个系统（或系统的一部分）。

【响应（Response）】该响应是在激励到达后所采取的行动。

【响应度量（Measurement）】当响应发生时，应当能够以某种方式对其进行度量，以对需求进行测试。



软件架构评估－架构评估方法

希赛

【可用性质量属性场景描述】

场景要素	可能的情况
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常操作、降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件、并进行如下一个或多个活动： 将其记录下来通知适当的各方，包括用户和其他系统； 根据已定义的规则禁止导致错误或故障的事件源 在一段预先指定的时间间隔内不可用，其中时间间隔取决于系统的关键程度在正常或降级模式下运行
响应度量	系统必须可用的时间间隔 可用时间 系统可以在降级模式下运行的时间间隔 故障修复时间



软件架构评估－架构评估方法

希赛

【可修改性质量属性场景描述】

场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会影响其他功能，对所做的修改进行测试部署所做的修改
响应度量	根据所影响元素的数量度量的成本、努力、资金；该修改对其他功能或质量属性所造成影响的程度



软件架构评估－架构评估方法

希赛

【性能质量属性场景描述】

场景要素	可能的情况
刺激源	用户请求，其他系统触发等
刺激	定期事件到达、随机事件到达、偶然事件到达
环境	正常模式、超载 (Overload) 模式
制品	系统
响应	处理刺激、改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率、数据丢失率



软件架构评估－架构评估方法

希赛

【可测试性质量属性场景描述】

场景要素	可能的情况
刺激源	开发人员、增量开发人员、系统验证人员、客户验收测试人员、系统用户
刺激	已完成的分析、架构、设计、类和子系统集成；所交付的系统
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、完整的应用
响应	提供对状态值的访问，提供所计算的值，准备测试环境
响应度量	已执行的可执行语句的百分比 如果存在缺陷出现故障的概率 执行测试的时间 测试中最长依赖的长度 准备测试环境的时间



软件架构评估－架构评估方法

希赛

【易用性质量属性场景描述】

场景要素	可能的情况
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统、使错误的影响最低、适配系统、对系统满意
环境	系统运行时或配置时
制品	系统
响应	(1)系统提供以下一个或多个响应来支持“学习系统特性”：帮助系统与环境联系紧密界面为用户所熟悉；在不熟悉的环境中，界面是可以使用的 (2)系统提供以下一个或多个响应来支持“有效使用系统”：数据和(或)命令的聚合；已输入的数据和(或)命令的重用；支持在界面中的有效导航； 具有一致操作的不同视图；全面搜索；多个同时进行的活动 (3)系统提供以下一个或多个响应来“使错误的影响最低”：撤销；取消从系统故障中恢复；识别并纠正用户错误；检索忘记的密码；验证系统资源 (4)系统提供以下一个或多个响应来“适配系统”：定制能力；国际化 (5)系统提供以下一个或多个响应来使客户“对系统的满意”：显示系统状态；与客户的节奏合拍
响应度量	任务时间、错误数量、解决问题的数量、用户满意度、用户知识的获得、成功操作在总操 作中所占的比例、损失的时间/丢失的数据量



软件架构评估－架构评估方法

希赛

【安全性质量属性场景描述】

场景要素	可能的情况
刺激源	正确识别、非正确识别身份未知的来自内部/外部的个人或系统；经过了授权/未授权它 访问了有限的资源/大量资源
刺激	试图显示数据，改变/删除数据，访问系统服务，降低系统服务的可用性
环境	在线或离线、联网或断网、连接有防火墙或者直接连到了网络
制品	系统服务、系统中的数据
响应	对用户身份进行认证；隐藏用户的身份；阻止对数据或服务的访问；允许访问数据或服务； 授予或收回对访问数据或服务的许可；根据身份记录访问、修改或试图访问修改数据、 服务；以一种不可读的格式存储数据；识别无法解释的对服务的高需求通知用户或另外一个系统，并限制服务的可用性
响应度量	用成功的概率表示，避开安全防范措施所需要的时间、努力、资源；检测到攻击的可能性； 确定攻击或访问、修改数据或服务的个人的可能性；在拒绝服务攻击的情况下仍然获得服务的百分比；恢复数据、服务；被破坏的数据、服务和(或)被拒绝的合法访问的范围



软件架构评估－架构评估方法

希赛

【可用性质量属性场景描述】

场景要素	可能的情况
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常操作、降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件、并进行如下一个或多个活动： 将其记录下来通知适当的各方，包括用户和其他系统； 根据已定义的规则禁止导致错误或故障的事件源 在一段预先指定的时间间隔内不可用，其中时间间隔取决于系统的关键程度在正常或降级模式下运行
响应度量	系统必须可用的时间间隔 可用时间 系统可以在降级模式下运行的时间间隔 故障修复时间

例：
系统能够连续运行的时间不小于1000小时，闪退后能够在15s之内自动重启



软件架构评估－架构评估方法

希赛

【可修改性质量属性场景描述】

场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会影响其他功能，对所做的修改进行测试部署所做的修改
响应度量	根据所影响元素的数量度量的成本、努力、资金；该修改对其他功能或质量属性所造成影响的程度

例：
更改Web界面接口必须在2人周内完成



软件架构评估－架构评估方法

希赛

【性能质量属性场景描述】

场景要素	可能的情况
刺激源	用户请求，其他系统触发等
刺激	定期事件到达、随机事件到达、偶然事件到达
环境	正常模式、超载 (Overload) 模式
制品	系统
响应	处理刺激、改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率、数据丢失率

例：
正常负载情况下，系统必须在0.5秒内对用户的交易请求进行响应



软件架构评估－架构评估方法

希赛

【可测试性质量属性场景描述】

场景要素	可能的情况
刺激源	开发人员、增量开发人员、系统验证人员、客户验收测试人员、系统用户
刺激	已完成的分析、架构、设计、类和子系统集成；所交付的系统
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、完整的应用
响应	提供对状态值的访问，提供所计算的值，准备测试环境
响应度量	已执行的可执行语句的百分比 如果存在缺陷出现故障的概率 执行测试的时间 测试中最长依赖的长度 准备测试环境的时间

例：
测试人员在开发期间针对一个代码单元，执行一个测试序列，获取结果，并在20分钟内完成测试



软件架构评估－架构评估方法

希赛

【易用性质量属性场景描述】

场景要素	可能的情况
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统、使错误的影响最低、适配系统、对系统满意
环境	系统运行时或配置时
制品	系统
响应	(1)系统提供以下一个或多个响应来支持“学习系统特性”：帮助系统与环境联系紧密界面为用户所熟悉；在不熟悉的环境中，界面是可以使用的 (2)系统提供以下一个或多个响应来支持“有效使用系统”：数据和(或)命令的聚合；已输入的数据和(或)命令的重用；支持在界面中的有效导航； 具有一致操作的不同视图；全面搜索；多个同时进行的活动 (3)系统提供以下一个或多个响应来“使错误的影响最低”：撤销；取消从系统故障中恢复；识别并纠正用户错误；检索忘记的密码；验证系统资源 (4)系统提供以下一个或多个响应来“适配系统”：定制能力；国际化 (5)系统提供以下一个或多个响应来使客户“对系统的满意”：显示系统状态；与客户的节奏合拍
响应度量	任务时间、错误数量、解决问题的数量、用户满意度、用户知识的获得、成功操作在总操 作中所占的比例、损失的时间/丢失的数据量

例：
新用户学习使用系统
时间不超过2小时



软件架构评估－架构评估方法

希赛

【安全性质量属性场景描述】

场景要素	可能的情况
刺激源	正确识别、非正确识别身份未知的来自内部/外部的个人或系统；经过了授权/未授权它 访问了有限的资源/大量资源
刺激	试图显示数据，改变/删除数据，访问系统服务，降低系统服务的可用性
环境	在线或离线、联网或断网、连接有防火墙或者直接连到了网络
制品	系统服务、系统中的数据
响应	对用户身份进行认证；隐藏用户的身份；阻止对数据或服务的访问；允许访问数据或服务； 授予或收回对访问数据或服务的许可；根据身份记录访问、修改或试图访问修改数据、 服务；以一种不可读的格式存储数据；识别无法解释的对服务的高需求通知用户或另外一个系统，并限制服务的可用性
响应度量	用成功的概率表示，避开安全防范措施所需要的时间、努力、资源；检测到攻击的可能性； 确定攻击或访问、修改数据或服务的个人的可能性；在拒绝服务攻击的情况下仍然获得服务的百分比；恢复数据、服务；被破坏的数据、服务和(或)被拒绝的合法访问的范围

例：
信用卡支付必须保证99.999%的安全性



软件架构评估 – 基于场景的评估方法

希赛

【软件架构分析法（SAAM）】

最初关注可修改性，后扩充到可移植性、可扩充性等。

最早形成文档并得到广泛使用的软件架构分析方法。

【架构权衡分析法（ATAM）】

由SAAM发展而来，主要针对：性能、实用性、安全性、可修改性。

【成本效益分析法（CBAM）】

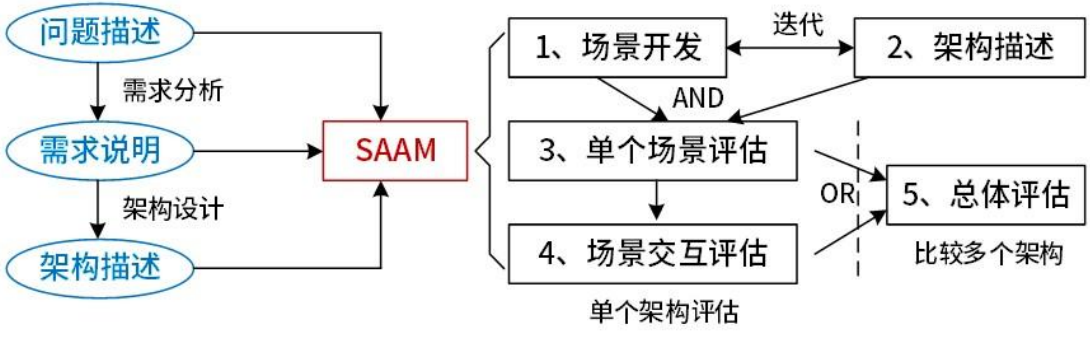
在ATAM基础上建立的，软件的“经济”模型。



软件架构评估 - SAAM

希赛

SAAM：最初用于分析架构可修改性，后扩展到其他质量属性。

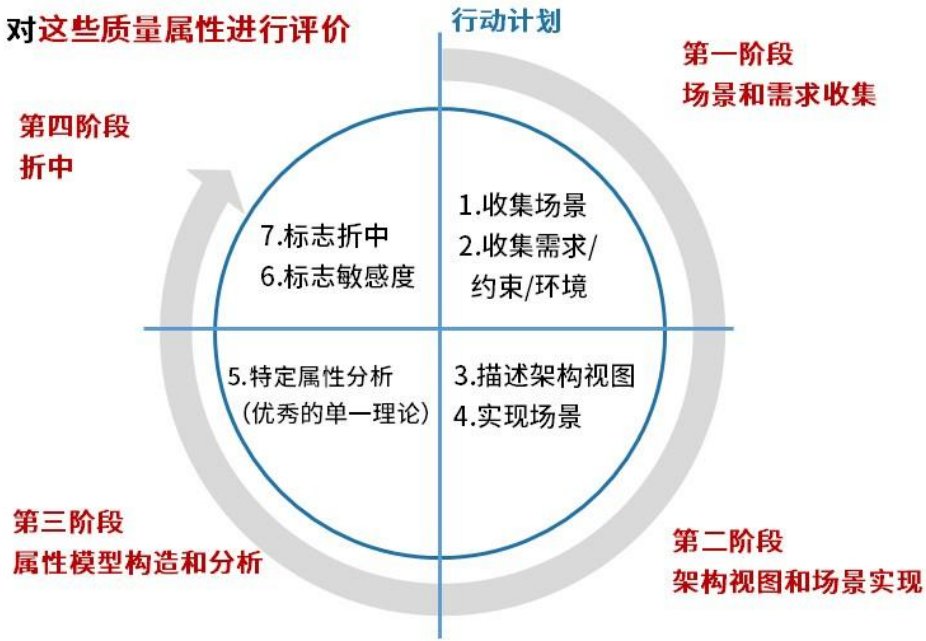




软件架构评估 - ATAM

希赛

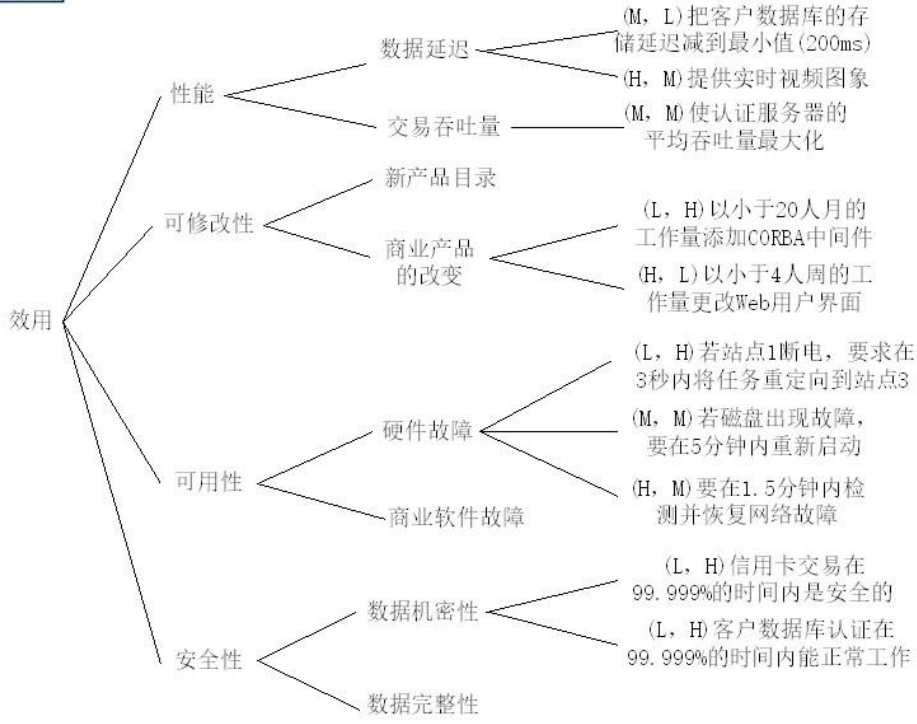
ATAM：在SAAM的基础上发展起来的，主要针对性能、实用性、安全性和可修改性，在系统开发之前，对这些质量属性进行评价和折中。





软件架构评估－质量效用树

希赛





软件架构评估

识别风险、非风险、敏感点和权衡点是进行软件架构评估的重要过程。“改变业务数据编码方式会对系统的性能和安全性产生影响”是对（ ）的描述，“假设用户请求的频率为每秒1个，业务处理时间小于30毫秒，则将请求响应时间设定为1秒钟是可以接受的”是对（ ）的描述。

A 风险点　　B 非风险　　C 敏感点　　D 权衡点

A 风险点　　B 非风险　　C 敏感点　　D 权衡点



软件架构评估

希赛

Architecture Tradeoff Analysis Method（ATAM）是一种软件架构的评估方法，以下关于该方法的叙述中，正确的是（ ）。

A ATAM是一种代码评估方法
B ATAM需要评估软件的需求是否准确
C ATAM需要对软件系统进行测试
D ATAM不是一种精确的评估工具



软件架构评估

体系结构权衡分析方法（Architecture Tradeoff Analysis Method，ATAM）包含4个主要的活动领域，分别是场景和需求收集、体系结构视图和场景实现、（ ）、折中。基于场景的架构分析方法（Scenarios-based Architecture Analysis Method，SAAM）的主要输入是问题描述、需求声明和（ ）。

A 架构设计　　B 问题分析与建模
C 属性模型构造和分析　　D 质量建模
A 问题说明　　B 问题建模
C 体系结构描述　　D 需求建模



构件与中间件技术 – 概念

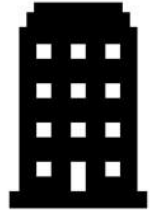
希赛

构件的定义

定义1：软件构件是一种组装单元，它具有规范的接口规约和显式的语境依赖。软件构件可以被独立地部署并由第三方任意地组装。

定义2：构件是某系统中有价值的、几乎独立的并可替换的一个部分，它在良好定义的体系结构语境内满足某清晰的功能。

定义3：构件是一个独立发布的功能部分，可以通过其接口访问它的服务。



模块的特性	⇨	**对象**的特性	⇨	**构件**的特性
结构化开发的产物		1、一个实例单元，具有唯一的标志； 2、可能具有状态，此状态外部可见； 3、封装了自己的状态和行为。		1、独立部署单元； 2、作为第三方的组装单元； 3、没有（外部的）可见状态。



构件与中间件技术 – 概念

希赛

构件系统架构特性

- 构件系统体系结构由一组平台决策、一组构件框架和构件框架之间的互操作设计组成。
- 构件框架是一种专用的体系结构（通常围绕一些关键的机制），同时，也是一组固定地作用于构件层次机制的策略。
- 概念框架的互操作设计包括系统体系结构连接的所有框架间的互操作的规则。
- 构件是一组通常需要同时部署的原子构件。构件和原子构件之间的区别在于，大多数原子构件永远都不会被单独部署，尽管它们可以被单独部署。
- 一个原子构件是一个模块和一组资源。
- 模块是一组类和可能的非面向对象的结构体，比如过程或者函数。
- 资源是一个类型化的项的固定集合。
- 资源这个概念可以包含代码资源，进而包含模块。问题在于除了编译器编译一个模块或包生成的资源外，还可能存在其他的资源。在“纯对象”的方法中，资源是外部化的不可改变的对象——不可改变是因为构件没有持久化的标志，而且复制不能被区分。



中间件

希赛

中间件是一类构件

中间件是一类系统软件 → 服务型软件.



ESB

简化结构、屏蔽差异、利于复用



中间件

希赛

采用中间件技术的优点

（1）面向需求。即设计师集中精力于业务逻辑本身。

（2）业务的分隔和包容性。应用开发人员可以按照不同的业务进行功能的划分，体现为不同的接口或交互模式。

（3）设计与实现隔离。构件对外发生作用或构件间的交互，都是通过接口进行的，构件使用者只需要知道构件的接口，而不必关心其内部实现，这是设计与实现分离的关键。

（4）隔离复杂的系统资源。架构很重要的一个功能就是将系统资源与应用构件隔离，这是保证构件可复用甚至“即插即用”的基础，与中间件的意图也是一致的。

（5）符合标准的交互模型。中间件则实现了架构的模型，实现了标准的协议。

（6）软件复用。中间件提供了构件封装、交互规则、与环境的隔离等机制，这些都为软件复用提供了方便的解决方案。

（7）提供对应用构件的管理。基于中间件的软件可以方便地进行管理，因为构件总可以通过标识机制进行划分。



构件与中间件技术 – 构件的复用

希赛

检索与提取构件

理解与评价构件

修改构件

组装构件

1、基于关键字的检索

特点：
树形或有向无回路图结构



2、刻面检索法

特点：
利用Facet描述构件执行的功能、被操作的数据、构件应用的语境或任意其他特征

例如：分多个刻面：
1、应用领域
2、使用环境
3、功能

3、超文本检索法

特点：
按照人类的联想思维方式任意跳转到包含相关概念或构件的文档



构件与中间件技术 – 构件的复用

希赛

检索与提取构件

理解与评价构件

修改构件

组装构件

1、要复用构件，准确地理解构件至关重要。特别是对构件修改使用时。

2、为达到目的，**必须要求构件的开发过程遵循公共标准**。

3、一般构件库的文档中全面而准确地说明以下内容：

构件的功能与行为、相关的领域知识、可适应性约束条件与例外情形、可以预见的修改部分及修改方法。



构件与中间件技术 – 构件的复用

希赛

检索与提取构件

理解与评价构件

修改构件

组装构件

1、**理想状态是直接复用构件库中现成的构件**，但大多数情况下，必须对构件进行或多或少的修改，以应对新需求。

2、为了减少构件修改的工作量，要求开发人员尽量使构件的功能、行为和接口设计更为抽象化、通用化和参数化。这样，复用者即可通过对实参的选取来调整构件的功能或行为。如果这种调整仍不足以使构件适用于新系统，复用者就必须借助设计信息和文档来修改构件。

3、构件库中若无可修改使用的构件，则按新需求开发构件，并存入构件库。



构件与中间件技术 – 构件的复用

希赛

检索与提取构件

理解与评价构件

修改构件

组装构件

组装的三种方式

1、基于功能的组装：采用子程序调用和参数传递的方式将构件组装起来。

2、基于数据的组装：仍然是传统的子程序调用与参数传递。但它所依赖的软件设计方法不再是功能分解，而是面向数据的设计方法，例如，Jackson系统开发方法。

3、面向对象的组装：如果从类库中检索出来的基类能够完全满足新系统的需求，则可以直接应用。否则，必须以基类为父类，生成相应的子类，以满足新系统的需求。

构件组装失配问题

1、由构件引起的失配，包括由于系统对构件基础设施、构件控制模型和构件数据模型的假设存在冲突引起的失配；

2、由连接子引起的失配，包括由于系统对构件交互协议、连接子数据模型的假设存在冲突引起的失配；

3、由于系统成分对全局体系结构的假设存在冲突引起的失配等。要解决失配问题，首先需要检测出失配问题，并在此基础上通过适当的手段消除检测出的失配问题。



构件与中间件技术 – 软件复用

希赛

软件重用可以分为垂直式重用和水平式重用，（　）是一种典型的水平式重用。

A 医学词汇表
B 标准函数库
C 电子商务标准
D 网银支付接口



构件与中间件技术 – 构件的复用

希赛

构件组装是指将库中的构件经适当修改后相互连接构成新的目标软件。（ ）不属于构件组装技术。

A 基于功能的构件组装技术
B 基于数据的构件组装技术
C 基于实现的构件组装技术
D 面向对象的构件组装技术



构件与中间件技术 – 构件的分类

希赛

从构件的外部形态来看，构件可分为5类：

（1）独立而成熟的构件。已在实际运行环境多次检验，该类构件隐藏了所有接口，用户只需用规定好的命令进行使用。例如，数据库管理系统和操作系统等。

（2）有限制的构件。有限制的构件提供了接口，指出了使用的条件和前提，这种构件在装配时，会产生资源冲突、覆盖等影响，在使用时需要加以测试。例如，各种面向对象程序设计语言中的基础类库等。

（3）适应性构件。适应性构件进行了包装或使用了接口技术，把不兼容性、资源冲突等进行了处理，可以直接使用。这种构件可以不加修改地使用在各种环境中。例如ActiveX等。

（4）装配的构件。装配（assemble）的构件在安装时，已经装配在操作系统、数据库管理系统或信息系统不同层次上，使用胶水代码（glue code）就可以进行连接使用。目前一些软件商提供的大多数软件产品都属这一类。

（5）可修改的构件。对原构件修改错误、增加新功能，可以利用重新“包装”或写接口来实现构件的版本替换。这种构件在应用系统开发中使用得比较多。



中间件

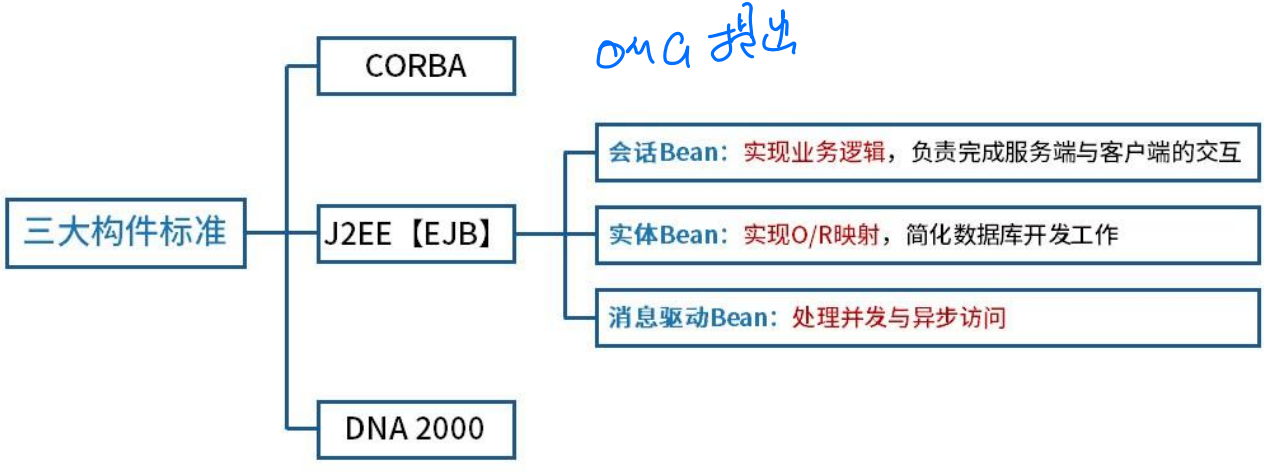
希赛

中间件分类	特点
通信处理（消息）中间件	可靠、高效、实时跨平台通信，eLink，MQSeries
事务处理（交易）中间件	事务分发，负载均衡，Tuxedo
数据存取管理中间件	为虚拟缓冲存取、格式转换、解压等带来方便
Web服务器中间件	有负载均衡、缓存、安全性等功能
安全中间件	如：加密，认证等
跨平台和架构的中间件	解决跨平台问题，如：CORBA
专用平台中间件	为特定应用领域设计领域参考模式，建立相应架构
网络中间件	功能包括网管、接入、网络测试、虚拟社区和虚拟缓冲等



构件与中间件技术 – 构件标准

希赛

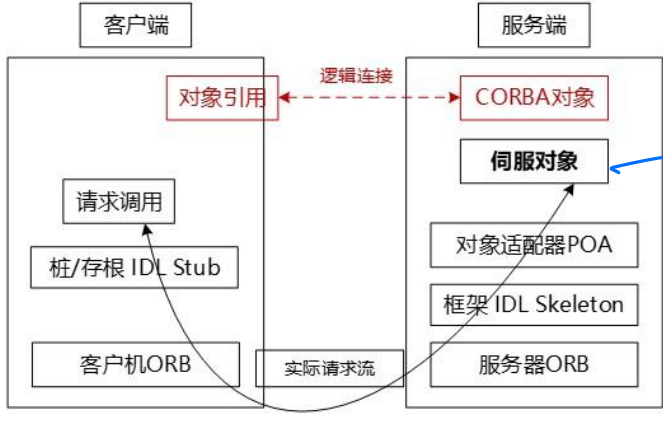


OMG 提出



构件与中间件技术 – CORBA

希赛



伺服对象（Servant）：CORBA对象的真正实现，负责完成客户端请求。

对象适配器（Object Adapter）：用于屏蔽ORB内核的实现细节，为服务器对象的实现者提供抽象接口，以便他们使用ORB内部的某些功能。

对象请求代理（Object Request Broker）：解释调用并负责查找实现该请求的对象，将参数传给找到的对象，并调用方法返回结果。客户方不需要了解服务对象的位置、通信方式、实现、激活或存储机制。



构件与中间件技术 – CORBA

希赛

CORBA服务端构件模型中，（　）是CORBA对象的真正实现，负责完成客户端请求。

A 伺服对象（Servant）
B 对象适配器（Object Adapter）
C 对象请求代理（Object Request Broker）
D 适配器激活器（Adapter Activator）



层次式架构设计理论与实践

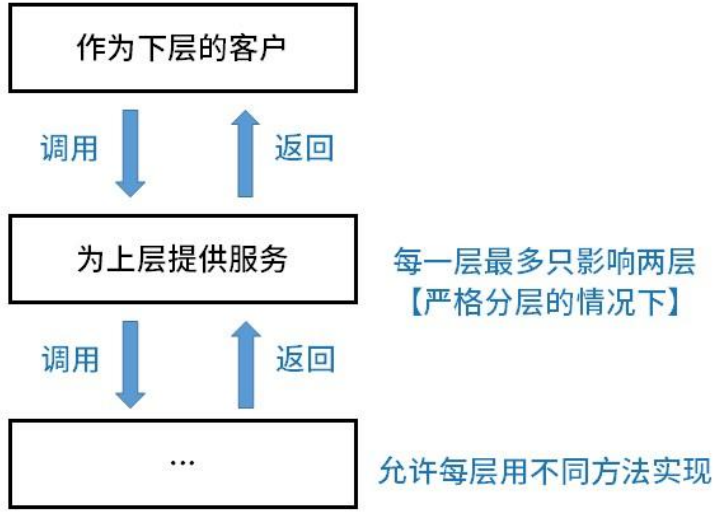
希赛

软件架构贯穿于软件研发的整个生命周期内，具有三方面的重要影响：

(1)利益相关人员之间的交流。

(2)系统设计的前期决策。

(3)可传递的系统级抽象。





C/S架构与B/S架构

希赛





升级维护困难

胖客户端
【客户应用程序】

【网络】

瘦服务器
【数据库服务器】



瘦客户端
【表示层】

应用服务器
【功能层】

数据库服务器
【数据层】

业务逻辑变化不再
需要更新客户端



零客户端
【Web浏览器/表示层】

WEB应用服务器
【功能层/数据访问层】

数据库服务器
【数据层】

完全不要客户端
升级维护容易



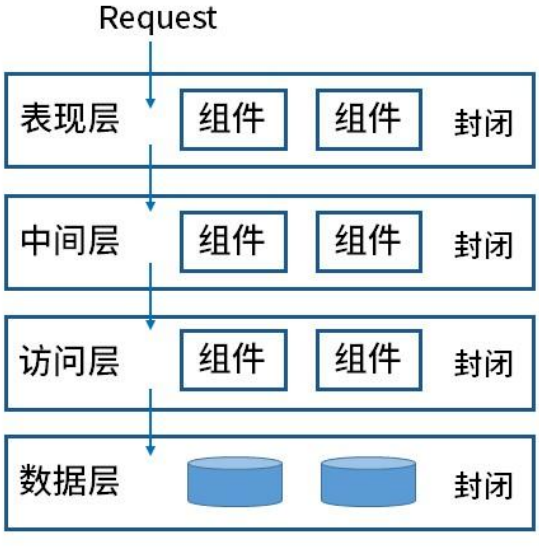
常用层次式架构

希赛

【N 层架构模式（n-tier architecture pattern）】

层次架构是最通用的架构，常作为初始架构

【关注分离】每层只负责本层的工作



MVC → MVP → MVVM　表现层设计模式

业务逻辑

ORM

【设计时注意两点】

1、污水池反模式。
请求简单穿透多层，但没做业务逻辑。

2、分层架构可能会让应用变得庞大。



MVC架构风格

希赛

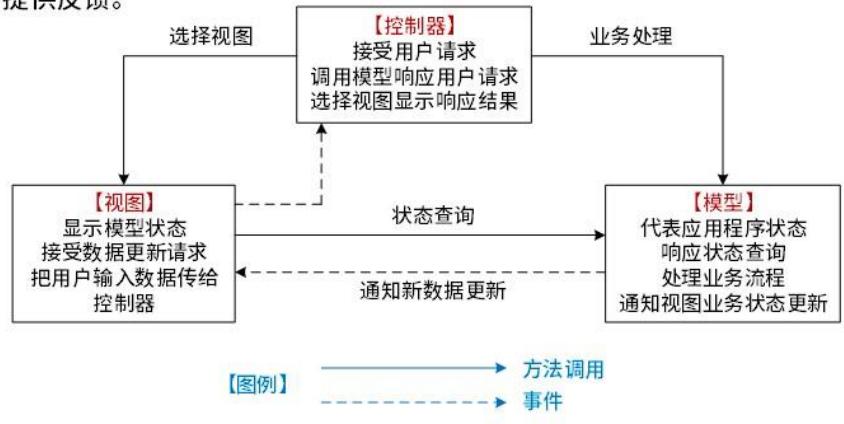
- **Model（模型）**：应用程序的主体部分。模型表示业务数据和业务逻辑。一个模型能为多个视图提供数据。提高应用的可重用性。
- **View（视图）**：用户看到并与之交互的界面。接收用户输入数据，向用户展示数据。
- **Controller（控制器）**：用户界面与Model的接口。解释视图的输入，将其解释为系统能够理解的对象，同时识别用户运作，将其解释为对模型特定方法的调用。处理来自于模型的事件和模型逻辑执行的结果，调用适当的视图为用户提供反馈。

★ J2EE体系结构中：

★ 视图（View）：JSP

★ 控制（Controller）：Servlet

★ 模型（Model）：Entity Bean、Session Bean



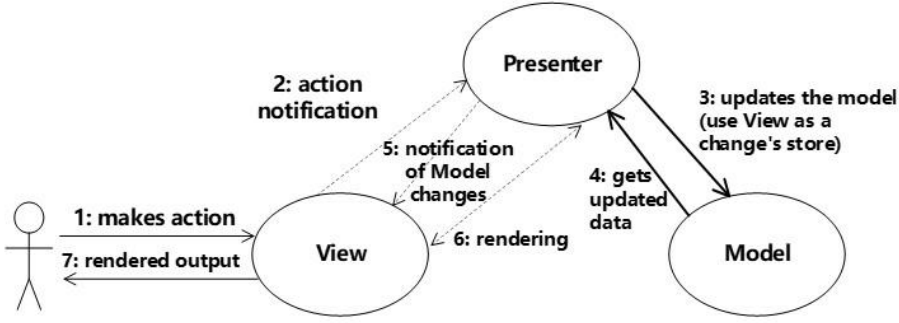


MVP架构风格

希赛

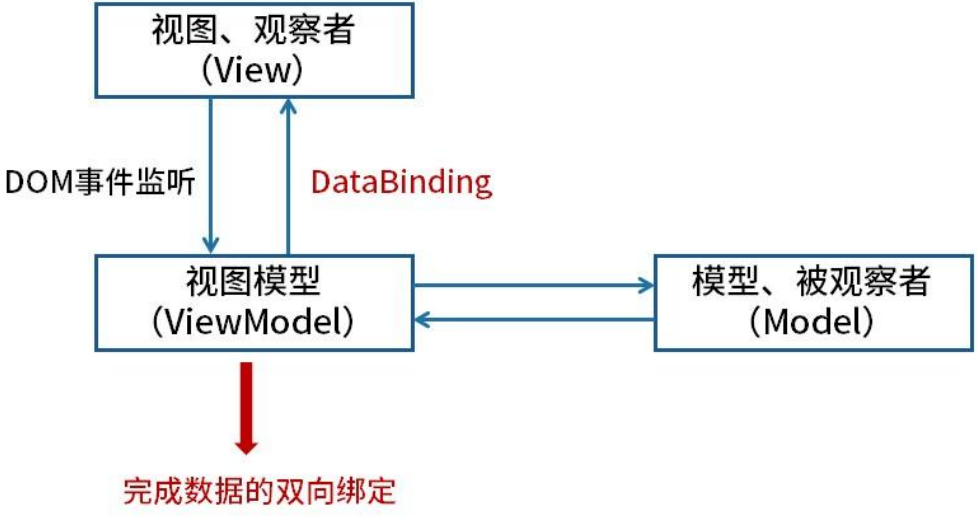
MVP是MVC的变种，其优点包括：

- ✓ 模型与视图完全分离，可以修改视图而不影响模型
- ✓ 可以更高效地使用模型，因为所有交互都发生在一个地方【Presenter】内部
- ✓ 可以将一个Presenter用于多个视图，而不需要改变Presenter的逻辑
- ✓ 如果把逻辑放在Presenter中，就可以脱离用户接口来测试这些逻辑（单元测试）





MVVM架构风格

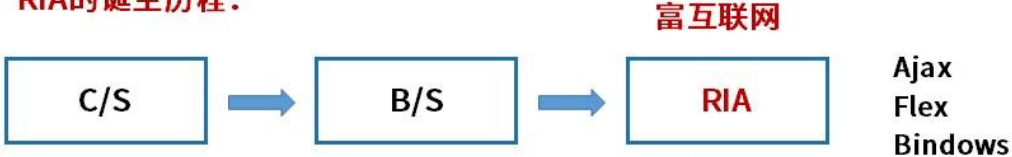

视图、观察者
(View)
DOM事件监听
DataBinding
视图模型
(ViewModel)
模型、被观察者
(Model)
完成数据的双向绑定



RIA架构风格

希赛

RIA的诞生历程：



【优点】

- ✓ 反应速度快
- ✓ 易于传播
- ✓ 交互性强



表现层UIP设计思想

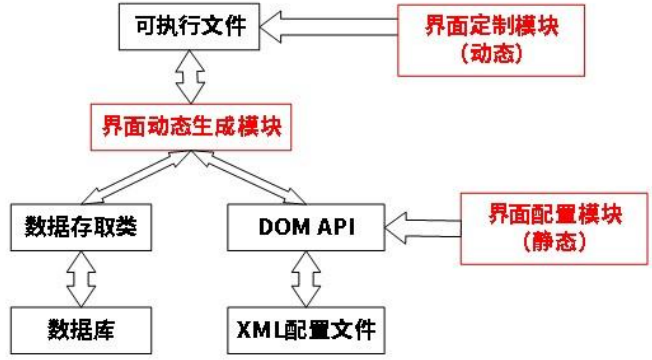
希赛

【UIP框架】

【User Interface Components】**原来的表现层**，用户看到的和进行交互都是这 个组件。
【User Interface Process Components】这个组件用于协调用户界面的各部分，使其配合后 台的活动，例如导航和工作流控制，以及状态和视图的管理。用户看不到这一组件，但是这些组件为User Interface Components提供了重要的支持功能。

【表现层动态生成设计思想】

基于XML界面管理技术，包括【界面配置】、【界面动态生成】和【界面定制】三部分。



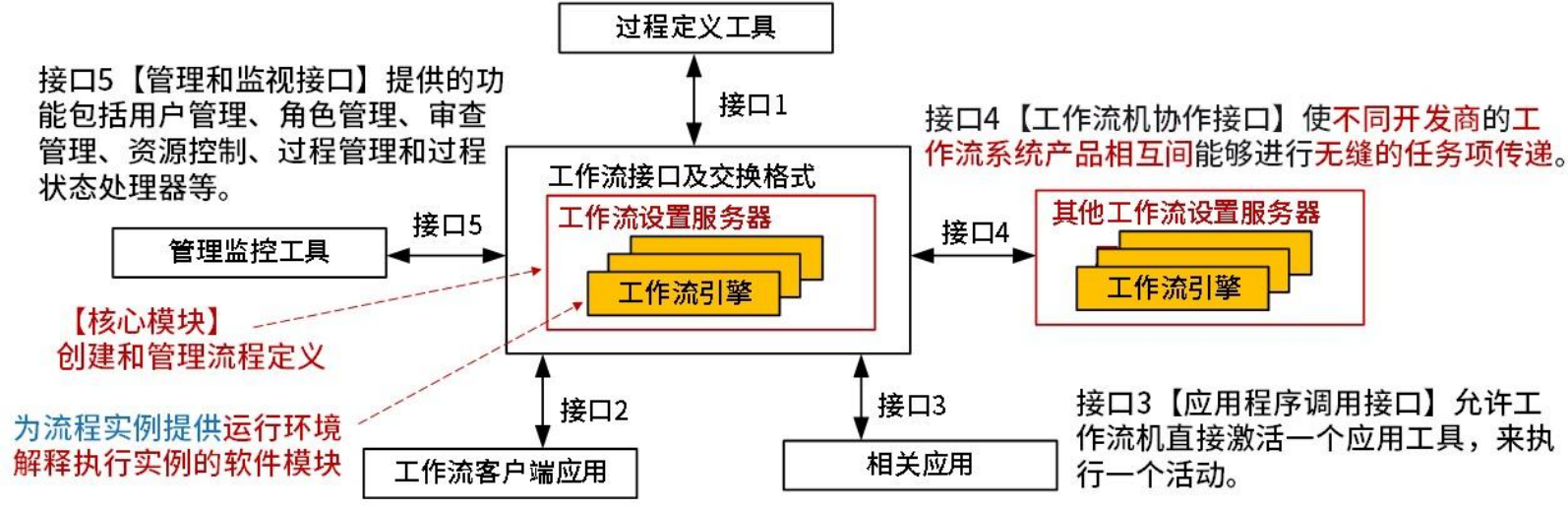


中间层

希赛

【业务逻辑层工作流设计】

接口1【过程定义导入/导出接口】转换格式和API调用，支持过程定义信息间的互相转换。



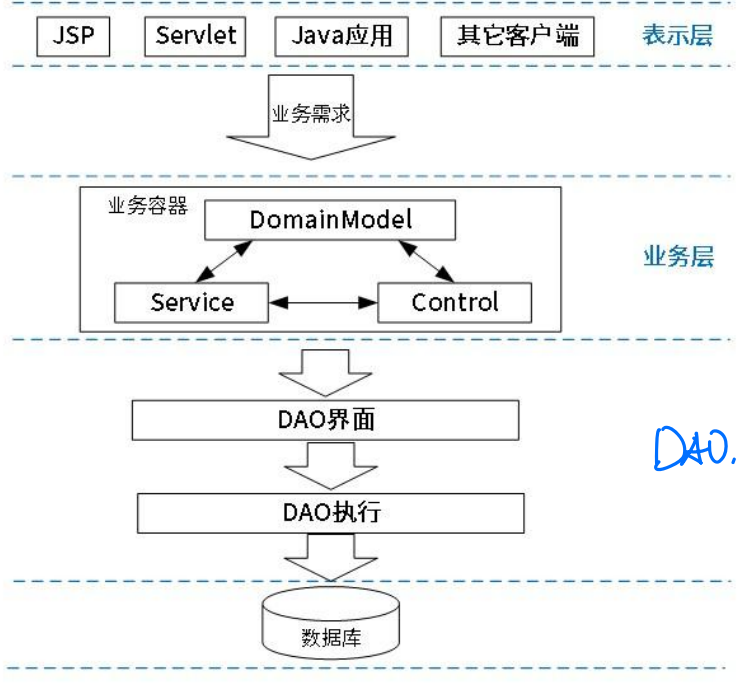
接口2【客户端应用程序接口】通过这个接口工作流机可以与任务表处理器交互，代表用户资源来组织任务，然后由任务表处理器负责，从任务表中选择、推进任务项，由任务表处理器或者终端用户来控制应用工具的活动。



中间层

希赛

【业务逻辑层框架】



MVC mvvm.

DAO. 数据访问层.



数据访问模式

希赛

5种数据访问模式

模式	特点
在线访问	每个数据库操作都会通过数据库连接不断地与后台的数据源进行交互
Data Access Object	底层数据访问操作与高层业务逻辑分离
Data Transfer Object	跨不同的进程或是网络的边界来传输数据
离线数据模式	从数据源获取数据后，存放到本地处理
对象/关系映射	应用系统中的对象与数据库中的数据表形成映射关系



数据访问层设计

希赛

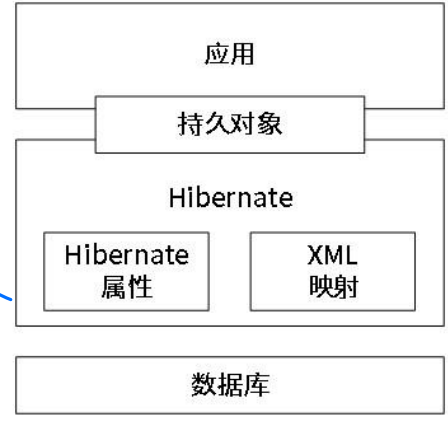
ORM（Object Relational Mapping）：对象与关系数据之间的映射。

映射关系表

面向对象	关系数据库
类（class）	数据库的表（table）
对象（object）	记录（record，行数据）
对象的属性（attribute）	字段（field）

实现技术对比表

维度	Hibernate	MyBatis（iBatis）
简单对比	强大，复杂，间接，SQL无关	小巧，简单，直接，SQL相关
可移植性	好（不关心具体数据库）	差（根据数据库SQL编写）
复杂多表关联	不支持	支持



Hibernate架构图



物联网分层架构

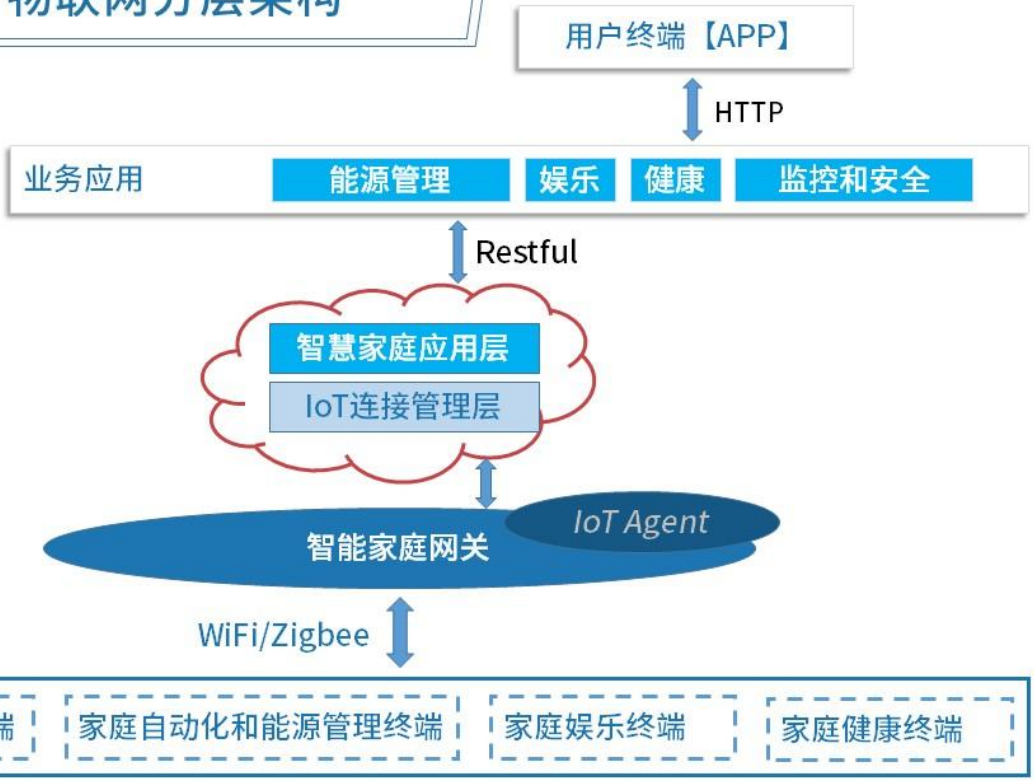
希赛

应用层	解决信息处理和人机交互问题 应用服务 智能终端
平台层	操作系统 软件开发 设备管理平台 连接管理平台
网络层	传递信息和处理信息 网络 通信标准/协议
感知层	解决数据获取问题 传感器 芯片 通信模组

物联网分层架构

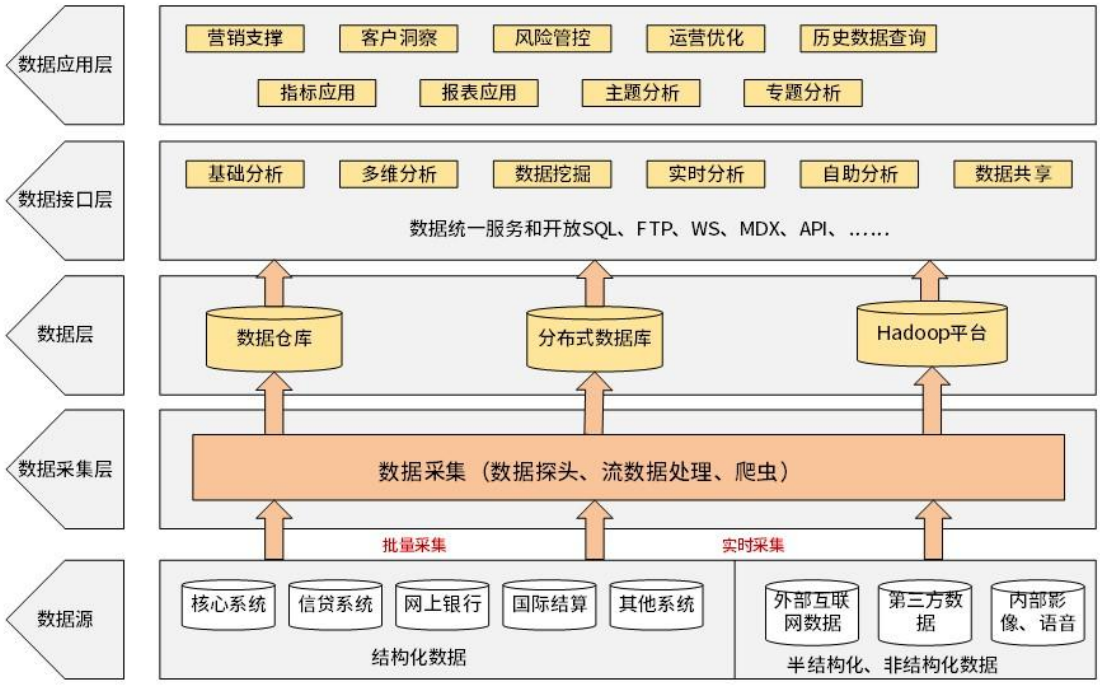


希赛


用户终端【APP】
HTTP
业务应用
能源管理
娱乐
健康
监控和安全
Restful
智慧家庭应用层
IoT连接管理层
IoT Agent
智能家庭网关
WiFi/Zigbee
监控和安全终端
家庭自动化和能源管理终端
家庭娱乐终端
家庭健康终端

大数据分层架构

大数据架构图

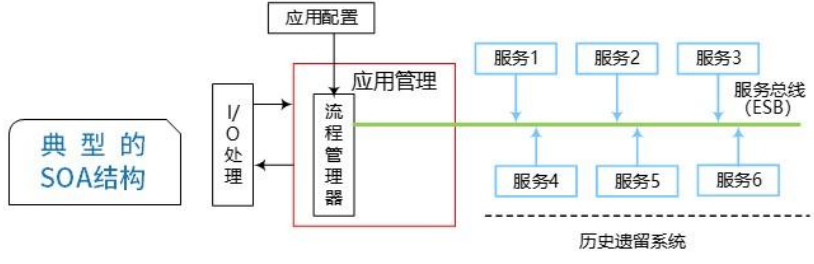




基于服务的架构（SOA）

希赛

服务是一种为了满足某项业务需求的操作、规则等的逻辑组合，它包含一系列有序活动的交互，为实现用户目标提供支持。







基于服务的架构（SOA）

希赛

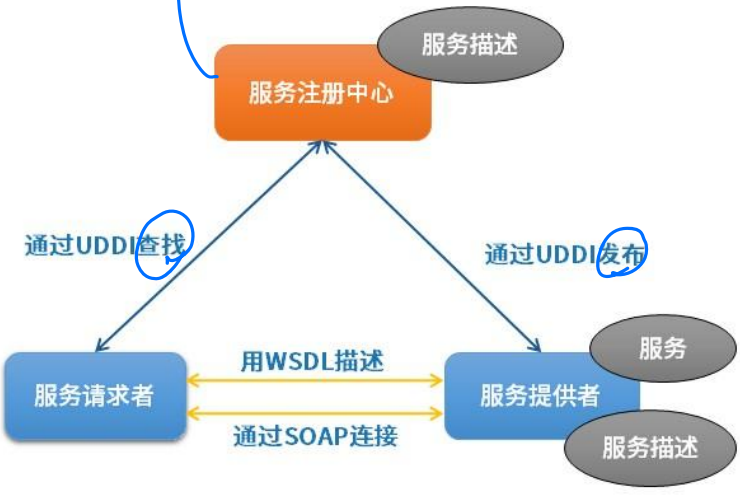


- ✓ 服务构件粗粒度，传统构件细粒度居多
- ✓ 服务构件的接口是标准的，主要是WSDL接口，传统构件常以具体API形式出现
- ✓ 服务构件的实现与语言无关，传统构件绑定某种特定语言
- ✓ 服务构件可以通过构件容器提供QoS的服务，传统构件完全由程序代码直接控制



Web Service（WEB服务）

希赛



功能	协议
发现服务	UDDI、DISCO
描述服务	WSDL、XML Schema
消息格式层	SOAP、REST
编码格式层	XML（DOM，SAX）
传输协议层	HTTP、TCP/IP 、SMTP等

WSDL就是WebService接口对应的WSDL文件，该文件通过xml格式说明如何调用，可以看作WebService的接口文档（使用说明书）。



REST（表述性状态转移）

希赛

REST（Representational State Transfer，表述性状态转移）是一种通常使用HTTP和XML进行基于Web通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

添加用户 GET /xisai_api/addUser	添加用户 POST /xisai_api/User
删除用户 GET /xisai_api/delUser	删除用户 DELETE /xisai_api/User
修改用户 GET /xisai_api/updateUser	修改用户 PUT /xisai_api/User

资源：如架构课程订单

表述：资源某一时间的状态，呈现形式有HTML、JSON、XML

状态转移：使用GET、POST等方法

超链接：通过超链接可与其它资源联系

REST的5个原则

（1）网络上的所有事物都被抽象为资源。

（2）每个资源对应一个唯一的资源标识（反过来不成立）。

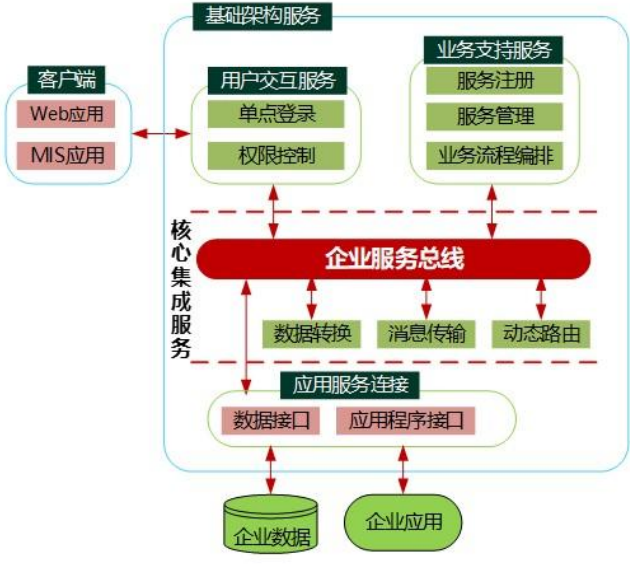
（3）通过通用的连接件接口对资源进行操作。

（4）对资源的各种操作不会改变资源标识。

（5）所有的操作都是无状态的。



ESB（企业服务总线）



服务请求者与服务提供者之间解耦

【ESB的核心功能】

- ✓ 提供位置透明性的消息路由和寻址服务
- ✓ 提供服务注册和命名的管理功能
- ✓ 支持多种消息传递范型
- ✓ 支持多种可以广泛使用的传输协议
- ✓ 支持多种数据格式及其相互转换
- ✓ 提供日志和监控功能

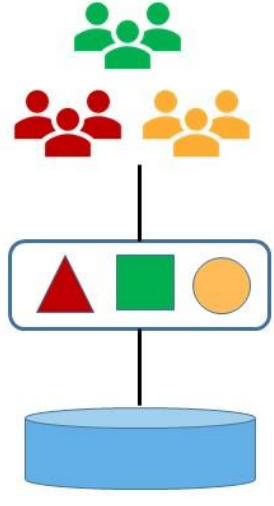


微服务

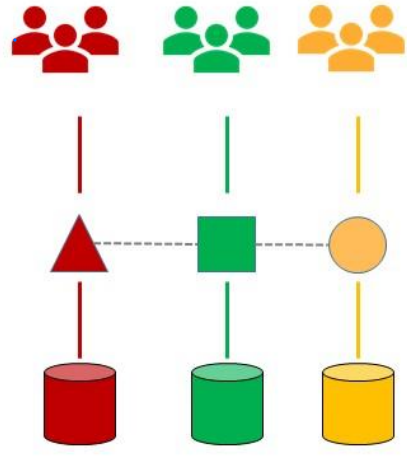
希赛

【微服务】顾名思义，就是很小的服务，所以它属于面向服务架构的一种。

雕版印刷 VS 活字印刷



【传统单体架构】



【微服务架构】



微服务

希赛

微服务的优点解读

优点	解读
【复杂应用解耦】	小服务（且专注于做一件事情） 化整为零，易于小团队开发
【独立】	独立开发 独立测试及独立部署（简单部署） 独立运行（每个服务独立在其独立进程中）
【技术选型灵活】	支持异构（如：每个服务使用不同数据库）
【容错】	故障被隔离在单个服务中，通过重试、平稳退化等机制实现应用层容错
【松耦合，易扩展】	可根据需求独立扩展

面临的问题与挑战【缺点】

- 分布式环境下的数据一致性【更复杂】
- 测试的复杂性【服务间依赖测试】
- 运维的复杂性

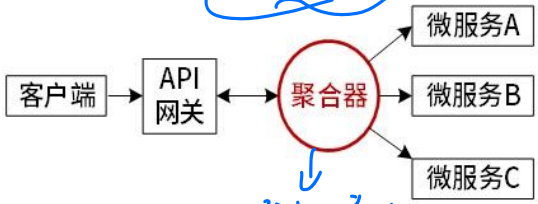


微服务

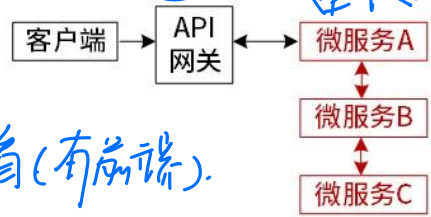
希赛

【微服务架构模式方案】

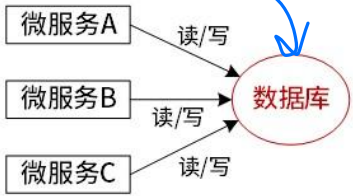
聚合器微服务



链式微服务



数据共享微服务



异步消息传递微服务





微服务与SOA

希赛

微服务	SOA
能拆分的就拆分	是整体的，服务能放一起的都放一起
纵向业务划分	是水平分多层
由单一组织负责	按层级划分不同部门的组织负责
细粒度	粗粒度
两句话可以解释明白	几百字只相当于SOA的目录
独立的子公司	类似大公司里面划分了一些业务单元（BU）
组件小	存在较复杂的组件
业务逻辑存在于每一个服务中	业务逻辑横跨多个业务领域
使用轻量级的通信方式，如HTTP	企业服务总线（ESB）充当了服务之间通信的角色

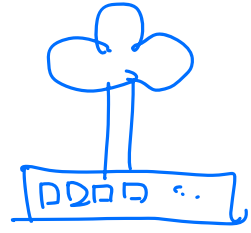


云计算概念/优点/分类

希赛

【云计算】是集合了大量计算设备和资源，对用户屏蔽底层差异的分布式处理架构，其用户与提供实际服务的计算资源是相分离的。

【云计算优点】超大规模、虚拟化、高可靠性、高可伸缩性、按需服务、成本低【前期投入低、综合使用成本也低】



云计算【按服务类型分类】

SaaS【软件即服务】	基于多租户技术实现，直接提供应用程序
PaaS【平台即服务】	虚拟中间件服务器、运行环境和操作系统
IaaS【基础设施即服务】	包括服务器、存储和网络等服务

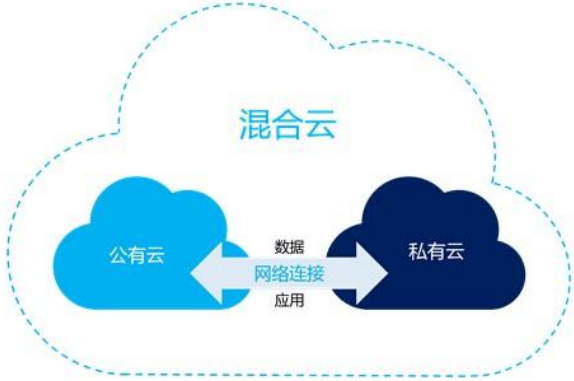


云计算概念/优点/分类

希赛

云计算【按部署方式分类】

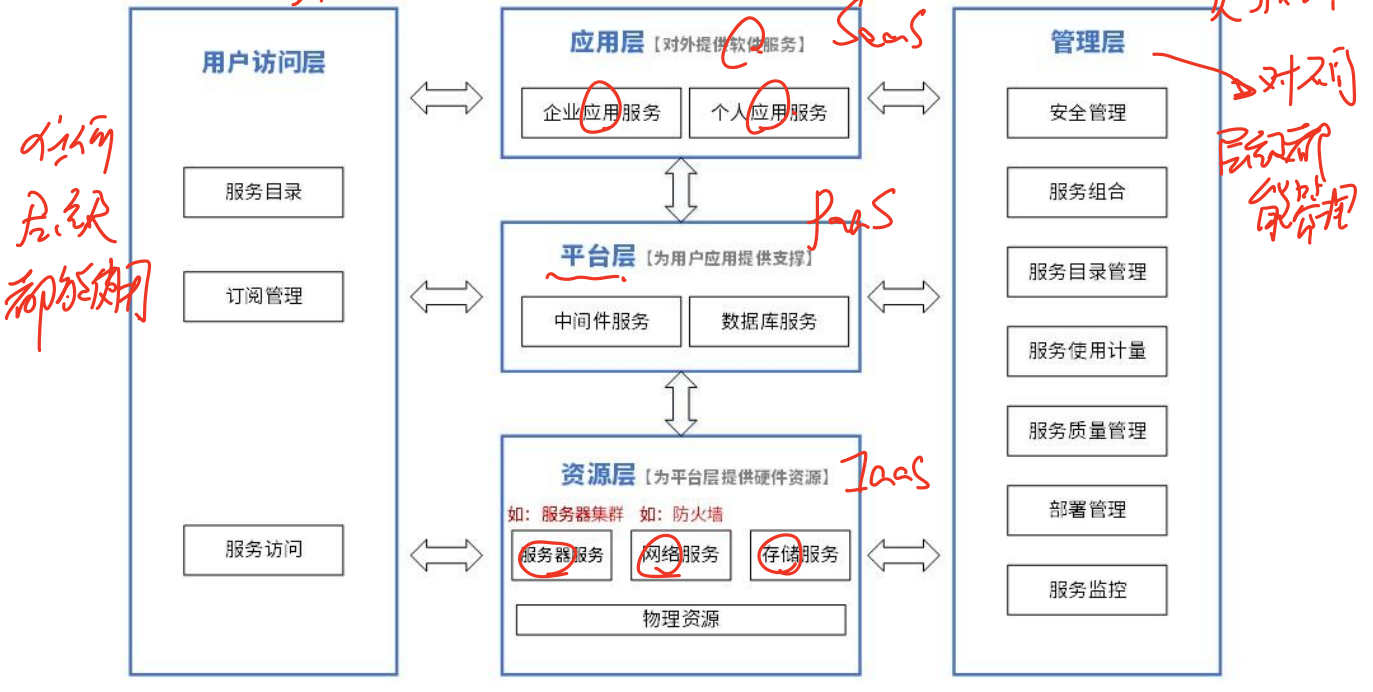
- 公有云：面向互联网用户需求，通过开放网络提供云计算服务
- 私有云：面向企业内部提供云计算服务
- 混合云：兼顾以上两种情况的云计算服务



云计算架构



希赛


用户访问层
服务目录
订阅管理
服务访问
应用层【对外提供软件服务】
企业应用服务
个人应用服务
SaaS
平台层【为用户应用提供支撑】
中间件服务
数据库服务
PaaS
资源层【为平台层提供硬件资源】
如：服务器集群
如：防火墙
服务器服务
网络服务
存储服务
物理资源
IaaS
管理层
安全管理
服务组合
服务目录管理
服务使用计量
服务质量管理
部署管理
服务监控



云计算架构

希赛

【管理层】提供对所有层次云计算服务的管理功能。

【用户访问层】方便用户使用云计算服务所需的各种支撑服务，针对每个层次的云计算服务都需要提供相应的访问接口。

【应用层】提供软件服务，如：财务管理，客户关系管理，商业智能。

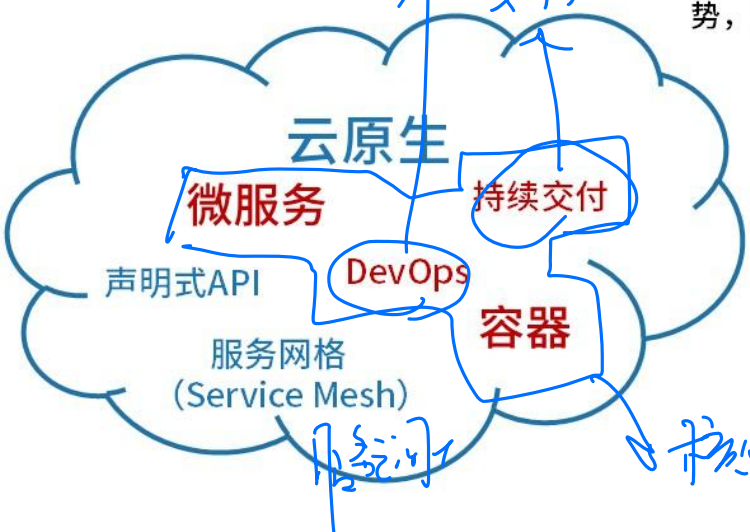
【平台层】为用户提供对资源层服务的封装，使用户可以构建自己的应用。

【资源层】提供虚拟化的资源，从而隐藏物理资源的复杂性。如：服务器，存储。



云原生架构

希赛



【云原生（Cloud Native）】一开始就是基于云环境、专门为云端特性而设计，可充分利用和发挥云平台的弹性+分布式优势，最大化释放云计算生产力。

【本质】

Oncloud ⇨ Incloud



云原生架构

希赛

【云原生架构设计原则】

- ✓ 服务化原则：使用微服务
- ✓ 弹性原则：可根据业务变化自动伸缩
- ✓ 可观测原则：通过日志、链路跟踪和度量
- ✓ 韧性原则：面对异常的抵御能力
- ✓ 所有过程自动化原则：自动化交付工具
- ✓ 零信任原则：默认不信任网络内部和外部的任何人/设备/系统
- ✓ 架构持续演进原则：业务高速迭代情况下的架构与业务平衡



云原生架构

希赛

【云原生架构模式】

1、服务化架构模式：典型代表【微服务】，服务拆分使维护压力大增。

2、Mesh化架构模式：把中间件框架（RPC、缓存、异步消息）从业务进程中分离，由Mesh进程完成。

3、Serverless模式：非常适合于事件驱动的数据计算任务。

4、存储计算分离模式：各类暂态数据（如session）用云服务保存。

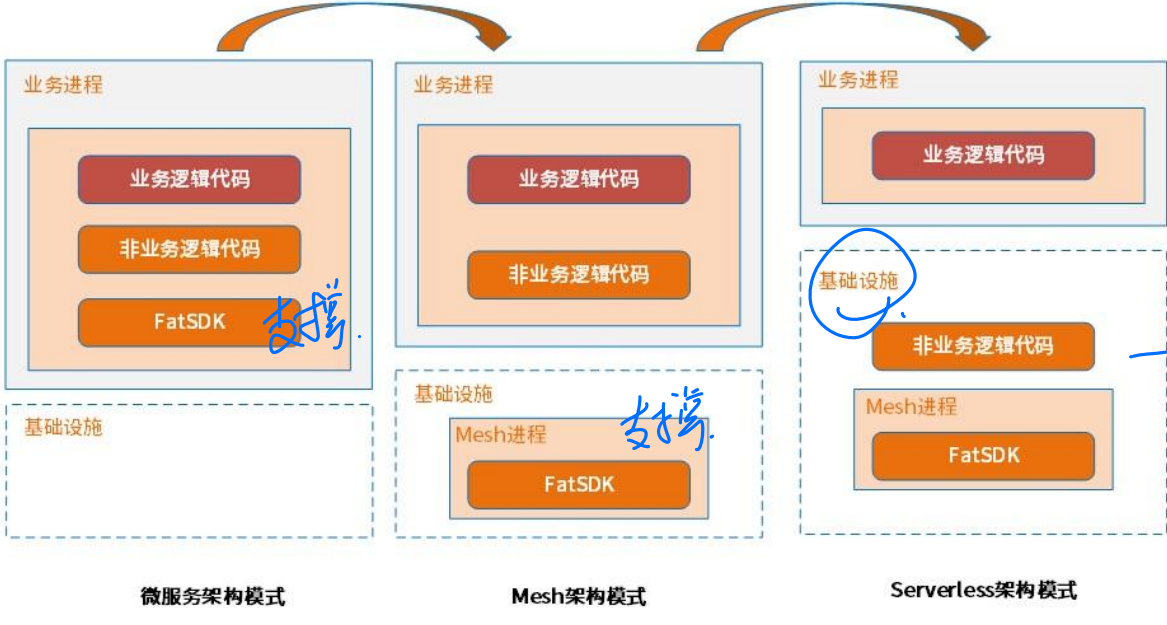
5、分布式事务模式：解决微服务模式中多数据源事务问题。

6、可观测架构：包括Logging、Tracing、Metrics三个方面。

7、事件驱动架构：本质上是一种应用/组件间的集成架构模式。



云原生架构



微服务架构模式　　Mesh架构模式　　Serverless架构模式



云原生架构

希赛

【云原生架构反模式】

1、庞大的单体应用

【需要多人开发的业务模块，考虑通过服务化进行拆分，并让组织与架构匹配】

2、单体应用“硬拆”为微服务（服务拆分要适度）

【小规模软件的服务拆分（为拆而拆）、数据依赖（服务间数据依赖）、性能降低】

3、缺乏自动化能力的微服务

【手动维护大量微服务是不现实的】



云原生架构

【容器技术】



传统部署模式　　虚拟化部署模式　　容器部署模式

Docker容器基于操作系统虚拟化技术，共享操作系统内核、轻量、没有资源损耗、秒级启动，极大提升了系统的应用部署密度和弹性。



云原生架构

希赛

【容器技术】

Kubernetes提供了**分布式应用管理的核心能力**。

【资源调度】根据请求资源量在集群中选择合适的节点来运行应用。

【应用部署与管理】支持应用的自动发布与应用的回滚。

【自动修复】当宿主机或者OS出现故障，节点健康检查会自动进行应用迁移。

【服务发现与负载均衡】结合DNS和负载均衡机制，支持容器化应用之间的相互通信。

【弹性伸缩】可以对这个业务进行自动扩容。

【声明式API】开发者可以关注于应用自身，而非系统执行细节。

【可扩展性架构】所有K8s组件都是基于一致的、开放的API实现和交互。

【可移植性】K8s通过一系列抽象如Load Balance Service（负载均衡服务）、CNI（容器网络接口）、CSI（容器存储接口），帮助业务应用可以屏蔽底层基础设施的实现差异，实现容器灵活迁移的设计目标。



云原生架构

希赛

【虚拟机技术与容器技术的对比】

对比项	虚拟机技术	容器技术
镜像大小	包含GuestOS，G量级以上	仅包含运行的Bin/Lib，M量级
资源要求	CPU与内存按核、按G分配	CPU与内存按单核、低于G量级分配
启动时间	分钟级	毫秒级
可持续性	跨物理机迁移	跨操作系统平台迁移
弹性伸缩	VM伸缩，CPU/内存手动伸缩	实例自动伸缩、CPU内存自动在线伸缩
隔离策略	操作系统、系统级别	Cgroups，进程级别



云原生架构

希赛

【微服务设计约束】

1、微服务个体约束

【每个微服务都是独立的，修改一个微服务不能影响另一个微服务】

2、微服务与微服务之间的横向关系

【通过第三方服务注册中心来满足服务的可发现性】

3、微服务与数据层之间的纵向约束

【数据是微服务的“私产”，访问时需要通过微服务】

4、全局视角下的微服务分布式约束

【高效运维整个系统】



云原生架构

希赛


应用
操作系统（Operating System）
工具与开发库（Utilities and Libraries）
Shell
GCC/Java
标准类库
配置管理
软件包管理 YUM等
API与系统服务（API and System Services）
POSIX
系统服务
Window 管理器
内核（OS Kernel）
System Calls
资源调度
进程管理
安全模型
I/O管理
设备驱动
安全管理服务
硬件
云原生应用
云原生操作系统（Cloud Native OS）
拓展服务
服务网络 Istio
AI计算 Kubeflow
无服务器 Knative
应用管理 OAM
镜像仓库
Kubernetes Resources
工作负载
存储卷
服务发现/路由
Kubernetes Core
Kubernetes Core
资源调度
Pod 管理
安全模型
CRD/Controller
基础设施资源抽象
存储CSI，网络CNI，Device Plugin，云供应商CCM等
安全管理服务
公共云、专有云、边缘计算



云原生架构

希赛

	传统开发	云原生开发
开发模式	瀑布模式，串行开发 设计 开发 测试 部署	水平切分，并行开发，更敏捷 设计 开发 测试 部署 设计 开发 测试 测试 设计 开发 测试 测试
技术架构	单体架构，耦合性强，灵活性差 UI Service\DB	纵向切分微服务，横向前后端分离。灵活性高 UI UI UI Service\DB Service\DB Service\DB
开发技能	JSP MVC JAVA spring	Vue MVVM JAVA springboot
团队协作	一个团队前后端都搞定 前端开发 后端开发	前后端两个团队， 约定接口，独立开发 前端开发 后端开发
部署	服务器安装部署全部自己搞，耗时费力，容易出错 应用服务器 文件服务器 数据库服务器 缓存服务器	服务开通即用，部署更快，稳定性高，省时省力 应用服务 文件服务 数据服务 缓存服务
运维	实体定位比较明确，对运维技能要求一般 应用运维 数据库运维 基础设施运维	SDX，比较抽象， 对运维技能要求高 云数据库 云基础设施 云应用



云原生架构

希赛

云计算可以按需提供弹性资源，它的体系结构由5部分组成。其中，（　）为用户提供对资源层的各项云计算服务的封装，帮助用户构建所需的应用。

A 应用层

B 平台层

C 用户访问层

D 管理层



云原生架构

希赛

Serverless将“部署”这个动作从运维中“收走”，把应用的整个运行都委托给云，这样能带来很多好处，以下（ ）场景比较适合Serverless架构模式。

A 应用是有状态的
B 应用会长时间在后台运行密集型计算任务
C 事件驱动的数据计算任务
D 应用涉及较多的外部I/O

边缘计算

希赛

【边缘计算】是指在靠近物或数据源头的一侧，采用网络、计算、存储、应用核心能力为一体的开放平台，就近提供最近端服务。

【边缘计算的本质】计算处理职能的本地化。

本地化处理.

【边缘计算类型】

【云边缘】云服务在边缘侧的延伸

【边缘云】在边缘侧构建中小规模云服务能力

【边缘网关】以云化技术与能力重构原有嵌入式网关系统





边缘计算

希赛

【云计算】



云计算中心

延迟：50

【大脑中枢纽】

擅长全局调度，进行非实时、长周期的大数据处理与分析

边一云协同.

【边缘计算】



边缘数据中心



边缘数据中心



边缘数据中心

延迟：10

【N个小脑】

侧重局部、小规模数据处理，适用于现场级、实时、短周期的智能分析与快速决策



边缘网关



边缘网关



边缘网关

延迟：5

【终端】



摄像头、汽车、手机、电脑、平板、传感器等各种终端

延迟：1

【无数神经末梢】

侧重局部、小规模数据处理，适用于现场级、实时、短周期的智能分析与快速决策

本地处理



边缘计算

希赛

【边云协同的分类】

【资源协同】边缘节点有基础设施资源的调度管理能力，可与云端协同。

【数据协同】边缘节点采集数据并初步分析，再发给云端做进一步处理。

【智能协同】分布式智能，云端做集中式模型训练，再将模型下发到边缘节点。

【应用管理协同】边缘节点提供应用部署与运行环境，云端主要提供应用开发、测试环境。

【业务管理协同】边缘节点提供模块化、微服务化的应用/数字孪生/网络等应用实例；云端主要提供按照客户需求实现应用/数字孪生/网络等的业务编排能力。

【服务协同】边缘节点按照云端策略实现部分ECSaaS服务，通过ECSaaS与云端SaaS的协同实现面向客户的按需SaaS服务；云端主要提供SaaS服务在云端和边缘节点的服务分布策略，以及云端承担的SaaS服务能力。



大型网站系统架构演化

希赛

高性能　高可用　可维护　应变　安全



大型网站系统架构演化

希赛

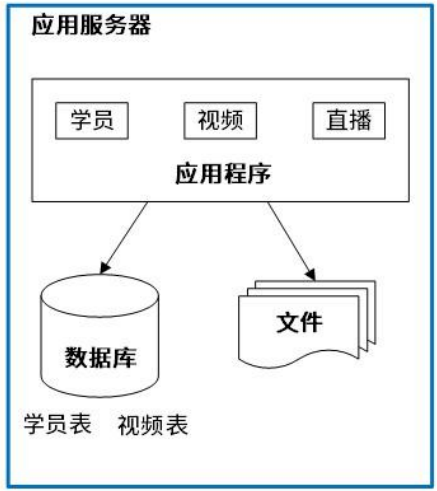
维度	涉及技术内容
从架构来看	MVC，MVP，MVVM，REST，Webservice，微服务
从并发分流来看	集群（负载均衡）、CDN
从缓存来看	MemCache，Redis，Squid
从数据来看	主从库（主从复制），内存数据库，反规范化技术，NoSQL，分区（分表）技术，视图与物化视图
从持久化来看	Hibernate，Mybatis
从分布存储来看	Hadoop，FastDFS，区块链
从数据编码来看	XML，JSON
从Web应用服务器来看	Apache，WebSphere，WebLogic，Tomcat，JBOSS，IIS
从安全性来看	SQL注入攻击
其他	静态化，有状态与无状态，响应式Web设计，中台



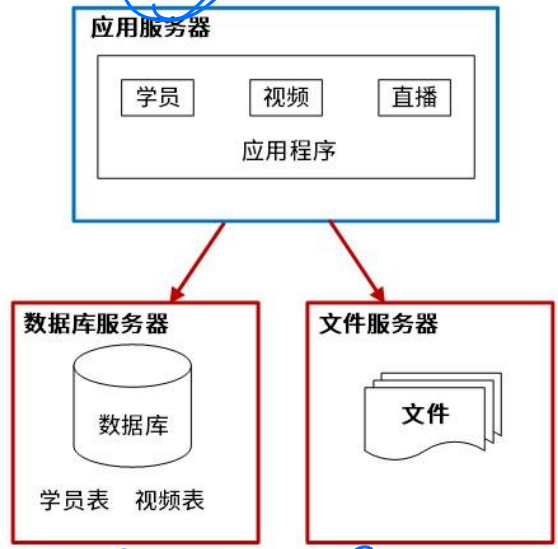
第一阶段：单体架构 到 第二阶段：垂直架构

希赛

第一阶段：单体架构



第二阶段：垂直架构

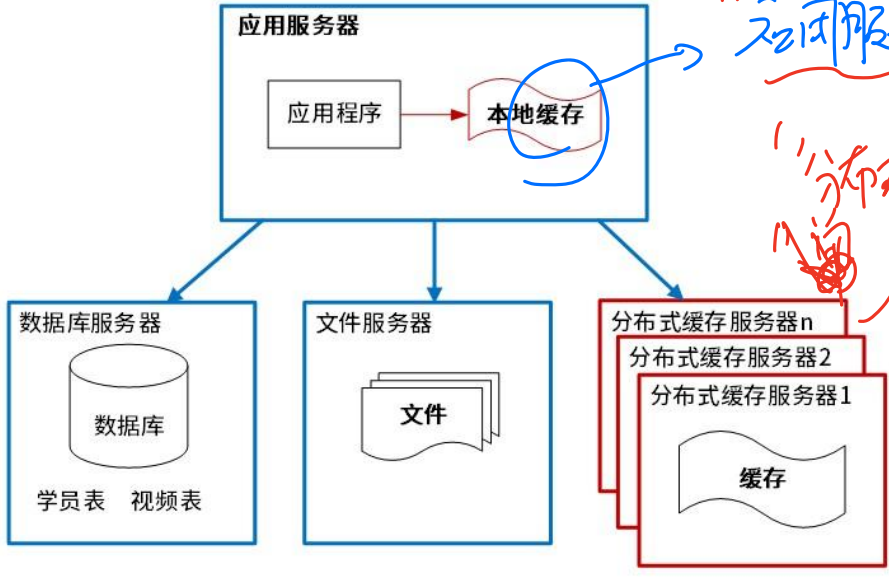


从1台服务器，调为3台服务器



第三阶段：使用缓存改善网站性能

希赛





第三阶段：使用缓存改善网站性能

希赛

常见缓存技术：

MemCache：Memcache是一个高性能的分布式的内存对象缓存系统，用于动态Web应用以减轻数据库负载。Memcache在内存里维护一个统一的巨大的hash表，数据存在该hash表中。

Redis：Redis是一个开源的使用ANSI C语言编写、支持网络、可基于内存亦可持久化的日志型、Key-Value数据库（支持多种数据结构，如key-value、string、set、list等），并提供多种语言的API。

Squid：Squid是一个高性能的代理缓存服务器，Squid支持FTP、gopher、HTTPS和HTTP协议。

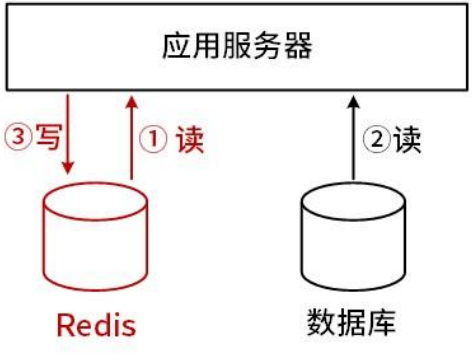


缓存与数据库的数据一致性问题

希赛

数据库与缓存数据是否有可能不一致？如何解决？

一、数据读取



操作步骤：

1、根据key从缓存读取

2、若缓存中没有，则根据key在数据库中查找

3、读取到“值”之后，更新缓存

代码示例：

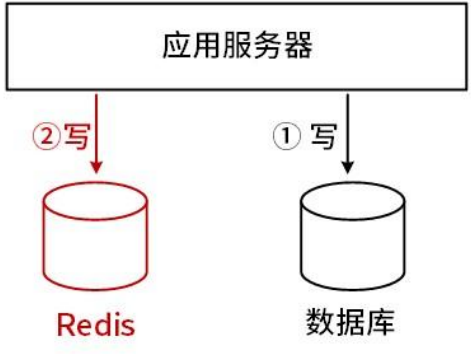
```
data = queryDataRedis(key);
if (data ==null) {
data = queryDataMySQL(key); //缓存查询不到，从MySQL做查询
if (data!=null) {
updateRedis(key, data);//查询完数据后更新到Redis
}
}
```



缓存与数据库的数据一致性问题

希赛

二、数据写入



没有完全统一的做法！

常见操作步骤：

1、根据key值写**数据库**

2、根据key更新缓存【或删除缓存】

代码示例：

```
updateMySQL(); //更新MySQL数据库
updateRedis(key, data); //更新缓存
```



缓存技术对比【MemCache与Redis】

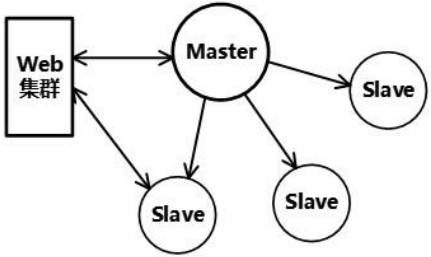
希赛

Redis 与 Memcache 能力比较

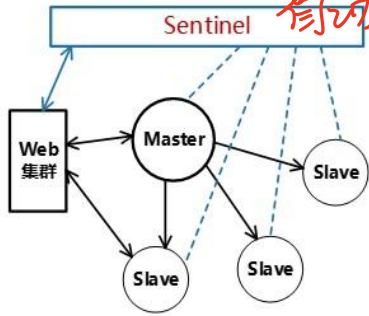
工作	MemCache	Redis
数据类型	简单key/value结构	丰富的数据结构
持久性	不支持	支持
分布式存储	客户端哈希分片/一致性哈希	多种方式，主从、Sentinel、Cluster等
多线程支持	支持	支持（Redis5.0及以前版本不支持）
内存管理	私有内存池/内存池	无
事务支持	不支持	有限支持
数据容灾	不支持，不能做数据恢复	支持，可以在灾难发生时，恢复数据

Redis分布式存储方案

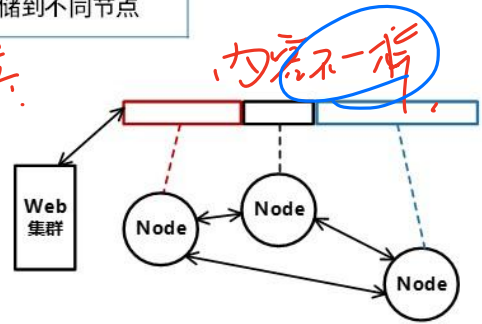
分布式存储方案	核心特点
主从（Master/Slave）模式	一主多从，故障时手动切换
哨兵（Sentinel）模式	有哨兵的一主多从，主节点故障自动选择新的主节点
集群（Cluster）模式	分节点对等集群，分slots，不同slots的信息存储到不同节点



主从（Master/Slave）模式



哨兵（Sentinel）模式



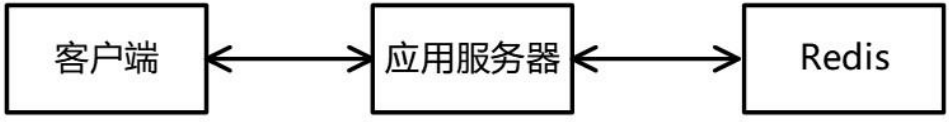
集群（Cluster）模式



Redis集群切片的常见方式

希赛

集群切片方式	核心特点
客户端分片	在客户端通过key的hash值对应到不同的服务器。
中间件实现分片	在应用软件和Redis中间，例如：Twemproxy、Codis等，由中间件实现服务到后台Redis节点的路由分派。
客户端服务端协作分片	Redis Cluster模式，客户端可采用一致性哈希，服务端提供节点的重定向到slot上。不同的slot对应到不同服务器。





Redis数据分片方案

希赛

分片方案	分片方式	说明
范围分片	按数据范围值来做分片	例：按用户编号分片，0-999999映射到实例A；1000000-1999999映射到实例B。
哈希分片	通过对key进行hash运算分片	可以把数据分配到不同实例，这类似于取余操作，余数相同的，放在一个实例上。
一致性哈希分片	哈希分片的改进	利于扩展节点，可以有效解决重新分配节点带来的无法命中问题。



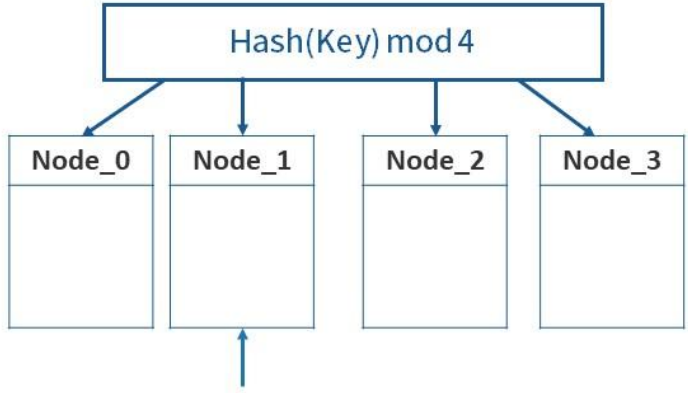
Redis数据分片方案

希赛

范围分片

Node_0	Node_1	Node_2	Node_3
ID<100000	ID<200000	ID<300000	其它

哈希分片

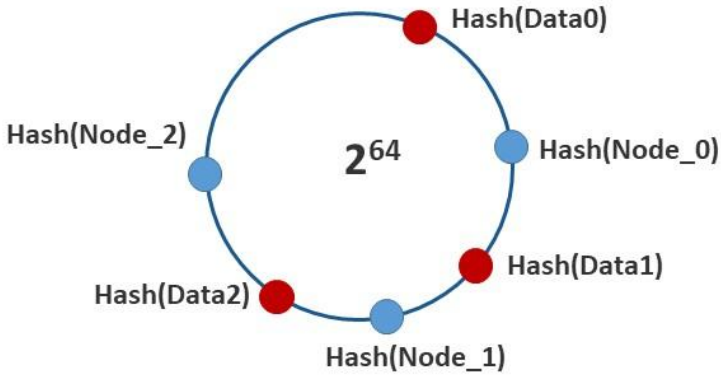




Redis数据分片方案

希赛

一致性哈希分片



64位的哈希



Redis 数据类型

希赛

类型	特点	示例
String（字符串）	存储二进制，任何类型数据，最大512MB	缓存，计数，共享Session
Hash（字典）	无序字典，数组+链表，适合存对象 Key对应一个HashMap。针对一组数据	存储、读取、修改用户属性
List（列表）	Linked List：双向链表，有序，增删快，查询慢 Array List：数组方式，有序，增删慢，查询快	消息队列，文章列表 记录前N个最新登录的用户ID列表
Set（集合）	键值对无序，唯一 增删查复杂度均为O(1)，支持交/并/差集操作	独立IP，共同爱好，标签
Sorted Set【Zset】（有序集合）	键值对有序，唯一，自带按权重排序效果	排行榜

Key	Value
User:1:name	jack
User:1:age	38

User:1

name	age
jack	38



Redis 数据淘汰算法

Redis

设置了过期时间

ttl 生存时间

希赛

淘汰作用范围	机制名	策略
不淘汰	noeviction	禁止驱逐数据，内存不足以容纳新入数据时，新写入操作就会报错。系统默认的一种淘汰策略。
设置了过期时间的键空间	volatile-random	随机移除某个key
	volatile-lru	优先移除最近未使用的key【局部性原理】
	volatile-ttl	ttl值小的key优先移除
全键空间	allkeys-random	随机移除某个key
	allkeys-lru	优先移除最近未使用的key



Redis的持久化

希赛

Redis的持久化主要有两种方式：RDB和AOF。

RDB：传统数据库中快照的思想。指定时间间隔将数据进行快照存储。

AOF：传统数据库中日志的思想，把每条改变数据集的命令追加到AOF文件末尾，这样出问题了，可以重新执行AOF文件中的命令来重建数据集。

对比维度	RDB持久化	AOF持久化
备份量	重量级的全量备份，保存整个数据库	轻量级增量备份，一次只保存一个修改命令
保存间隔时间	保存间隔时间长	保存间隔时间短，默认1秒
还原速度	数据还原速度快	数据还原速度慢
阻塞情况	save会阻塞，但bgsave或者自动不会阻塞	无论是平时还是AOF重写，都不会阻塞
数据体积	同等数据体积：小	同等数据体积：大
安全性	数据安全性：低，容易丢数据	数据安全性：高，根据策略决定

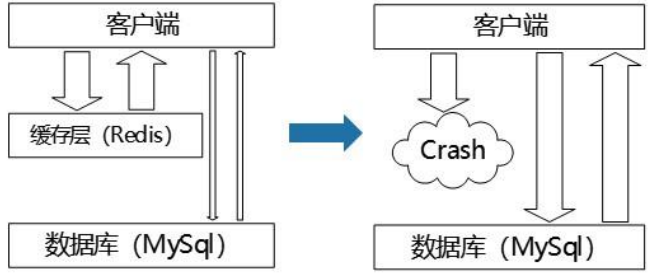


Redis常见问题

希赛

1、缓存雪崩

大部分缓存失效 -> 数据库崩溃



解决方案

1、使用锁或队列：保证不会有大量的线程对数据库一次性进行读写，从而避免失效时大量的并发请求落到底层存储系统上。

2、为key设置不同的缓存失效时间：在固定的一个缓存时间的基础上+随机一个时间作为缓存失效时间。

3、二级缓存：设置一个有时间限制的缓存+一个无时间限制的缓存。避免大规模访问数据库。



Redis常见问题

希赛

2、缓存穿透

查询无数据返回 -> 直接查数据库

解决方案

1、如果查询结果为空，直接设置一个默认值存放到缓存，这样第二次到缓冲中获取就有值了。设置一个不超过5分钟的过期时间，以便能正常更新缓存。

2、设置布隆过滤器，将所有可能存在的数据哈希到一个足够大的bitmap中，一个一定不存在的数据会被这个bitmap拦截掉，从而避免了对底层存储系统的查询压力。

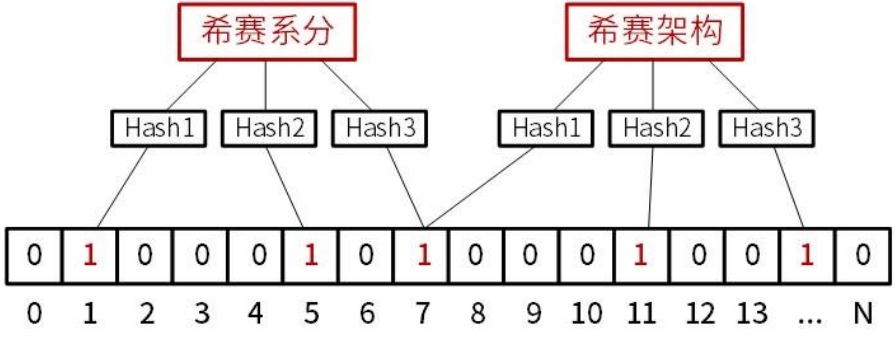


Redis常见问题

希赛

【布隆过滤器】 布隆过滤器用于快速识别1个元素不在一个集合中。

通过一个长二进制向量和一系列随机映射函数来记录与识别某个数据是否在一个集合中。



优点	缺点
1、占用内存小 2、查询效率高 3、不需要存储元素本身，在某些对保密要求比较严格的场合有很大优势	1、有一定的误判率，即存在假阳性，不能准确判断元素是否在集合中。 2、一般情况下不能从布隆过滤器中删除元素 3、不能获取元素本身



3、缓存预热

系统上线后，将相关需要缓存数据直接加到缓存系统中。

解决方案

1、直接写个缓存刷新页面，上线时手工操作。
2、数据量不大时，可以在项目启动的时候自动进行加载。
3、定时刷新缓存。

4、缓存更新

除Redis系统自带的缓存失效策略，常见采用以下两种：

1、定时清理过期的缓存。

2、当有用户请求过来时，再判断这个请求所用到的缓存是否过期，过期的话就去底层系统得到新数据并更新缓存。

5、缓存降级

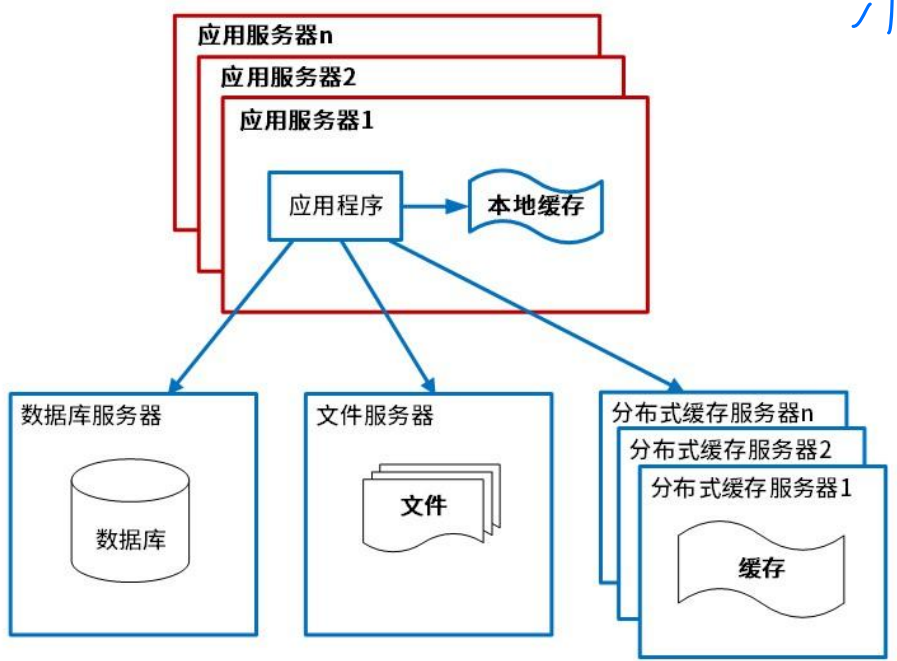
降级的目的是保证核心服务可用，即使是有损的，而且有些服务是无法降级的（如电商的购物流程等）；在进行降级之前要对系统进行梳理，从而梳理出哪些必须保护，哪些可降级。



第四阶段：使用服务集群改善网站并发处理能力

希赛

小型机，太贵。





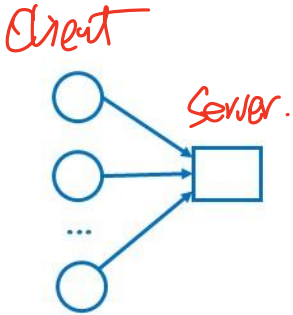
集群带来的问题

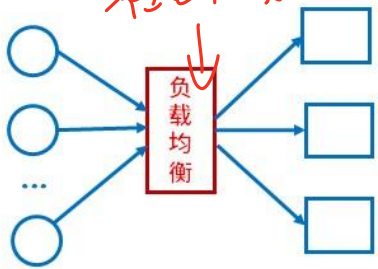
希赛

系统演变到这里，将会出现下面几个问题：

1、用户的请求由谁来转发到具体的应用服务器

2、用户如果每次访问到的服务器不一样，那么如何维护session的一致性





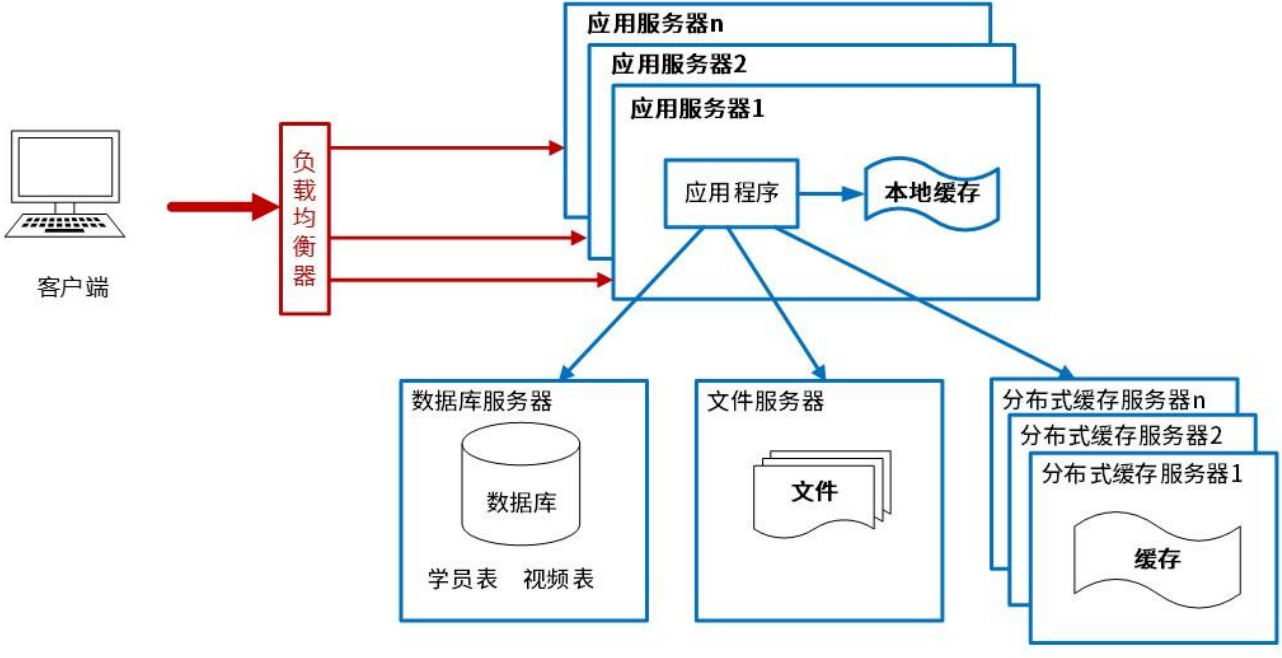
1、负载均衡

2、有状态与无状态问题



负载均衡的引入

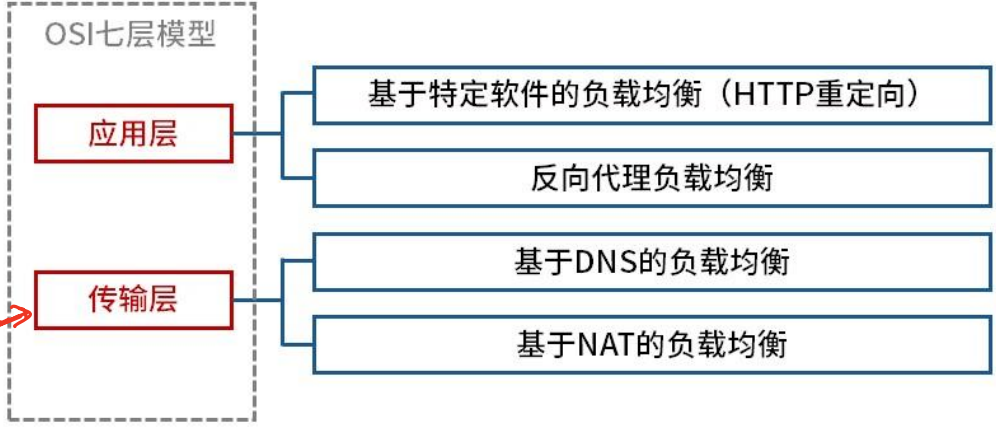
希赛


应用服务器n
应用服务器2
应用服务器1
应用程序
本地缓存
负载均衡器
客户端
数据库服务器
数据库
学员表 视频表
文件服务器
文件
分布式缓存服务器n
分布式缓存服务器2
分布式缓存服务器1
缓存



负载均衡技术

希赛


OSI七层模型
应用层
传输层
基于特定软件的负载均衡（HTTP重定向）
反向代理负载均衡
基于DNS的负载均衡
基于NAT的负载均衡



负载均衡技术

希赛

应用层负载均衡

1、http重定向。HTTP重定向就是应用层的请求转发。用户的请求其实已经到了HTTP重定向负载均衡服务器，服务器根据算法要求用户重定向，用户收到重定向请求后，再次请求真正的集群。

特点：实现简单，但性能较差。

2、反向代理服务器。在用户的请求到达反向代理服务器时（已经到达网站机房），由反向代理服务器根据算法转发到具体的服务器。常用的apache、nginx都可以充当反向代理服务器。

特点：部署简单，但代理服务器可能成为性能的瓶颈。



负载均衡技术

希赛

传输层负载均衡

1、DNS域名解析负载均衡。DNS域名解析负载均衡就是在用户请求DNS服务器，获取域名对应的IP地址时，DNS服务器直接给出负载均衡后的服务器IP。

特点：效率比HTTP重定向高，减少维护负载均衡服务器成本。但一个应用服务器故障，不能及时通知DNS，而且DNS负载均衡的控制权在域名服务商那里，网站无法做更多的改善和更强大的管理。

2、基于NAT的负载均衡。基于NAT的负载均衡将一个外部IP地址映射为多个IP地址，对每次连接请求动态地转换为一个内部节点的地址。

特点：技术较为成熟，一般在网关位置，可以通过硬件实现。像四层交换机一般就采用了这种技术。



负载均衡技术

希赛

◆ 静态算法（不考虑动态负载）

（1）轮转算法：轮流将服务请求（任务）调度给不同的节点（即：服务器）。

（2）加权轮转算法：考虑不同节点处理能力的差异。

（3）源地址哈希散列算法：根据请求的源IP地址，作为散列键从静态分配的散列表找出对应的节点。

（4）目标地址哈希散列算法：根据请求目标IP做散列找出对应节点。

（5）随机算法：随机分配，简单，但不可控。

◆ 动态算法（考虑动态负载）

（1）最小连接数算法：新请求分配给当前活动请求数量最少的节点，每个节点处理能力相同的情况下。

（2）加权最小连接数算法：考虑节点处理能力不同，按最小连接数分配。

（3）加权百分比算法：考虑了节点的利用率、硬盘速率、进程个数等，使用利用率来表现剩余处理能力。

◆ 硬件负载均衡：F5

◆ 软件负载均衡：LVS、Nginx、HAproxy



有状态与无状态

无状态服务（stateless service）对单次请求的处理，不依赖其他请求，也就是说，处理一次请求所需的全部信息，要么都包含在这个请求里，要么可以从外部获取到（比如说数据库），服务器本身不存储任何信息。

有状态服务（stateful service）则相反，它会在自身保存一些数据，先后的请求是有关联的。

判断以下构件是有状态服务还是无状态服务：

（1）Identification Bean（身份认证构件）

（2）ResPublish Bean（资源发布构件）

（3）ResRetrieval Bean（资源检索构件）

（4）OnlineEdit Bean（在线编辑构件）

（5）Statistics Bean（统计分析构件）

有状态与无状态（Session共享机制）



希赛



客户端

方案1：
携带session的cookie

负载均衡器

应用服务器1

应用程序

本地缓存

方案2：服务器间同步session

应用服务器2

应用程序

本地缓存

redis

方案3：将session存入redis

数据库服务器

数据库

文件服务器

文件

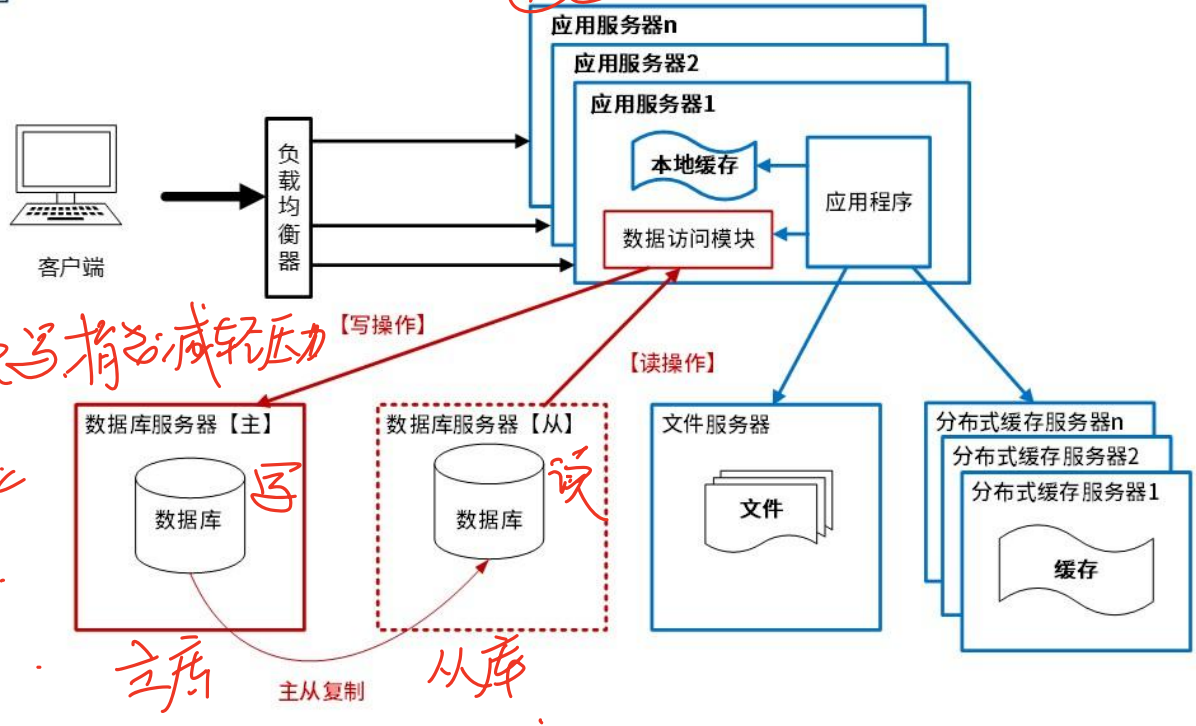
分布式缓存服务器n

分布式缓存服务器2

分布式缓存服务器1

缓存

第五阶段：数据库读写分离





第五阶段：数据库读写分离

希赛

主从数据库结构特点：

1、一般：一主多从，也可以多主多从。

2、主库做写操作，从库做读操作。

主从复制步骤：

1、主库（Master）更新数据完成前，将操作写binlog日志文件。

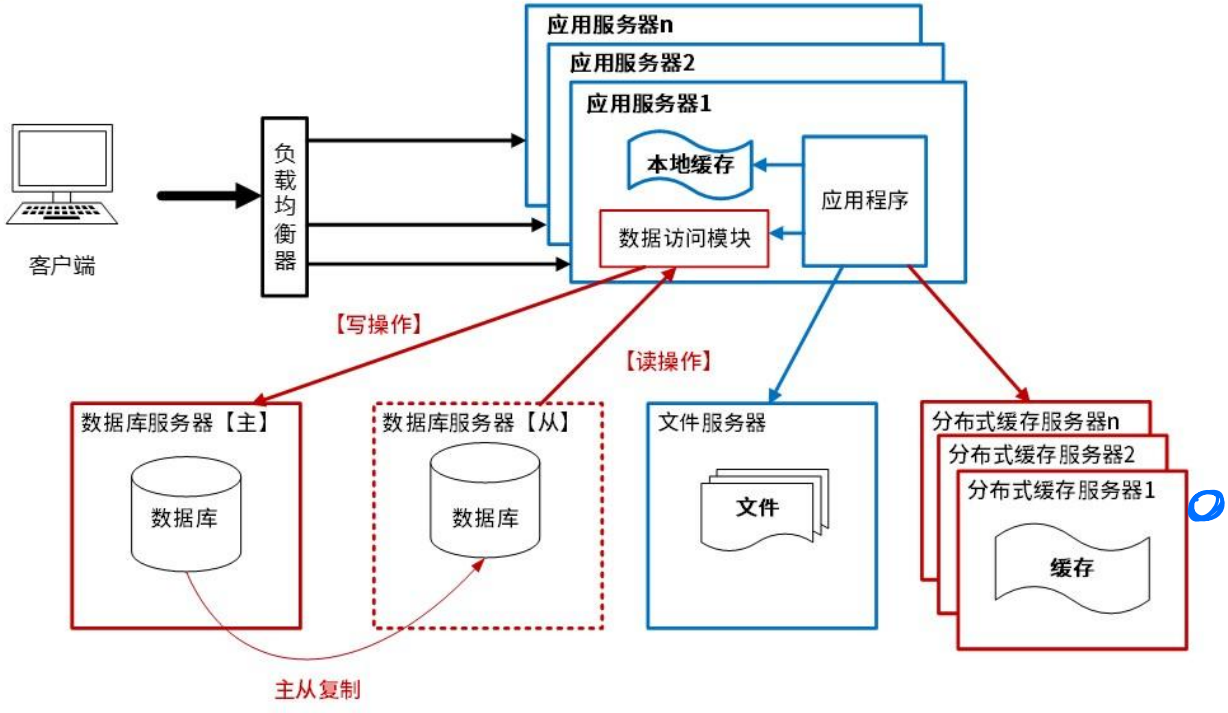
2、从库（Slave）打开I/O线程与主库连接，做binlog dump process，并将事件写入中继日志。

3、从库执行中继日志事件，保持与主库一致。



用缓存缓解读库的压力

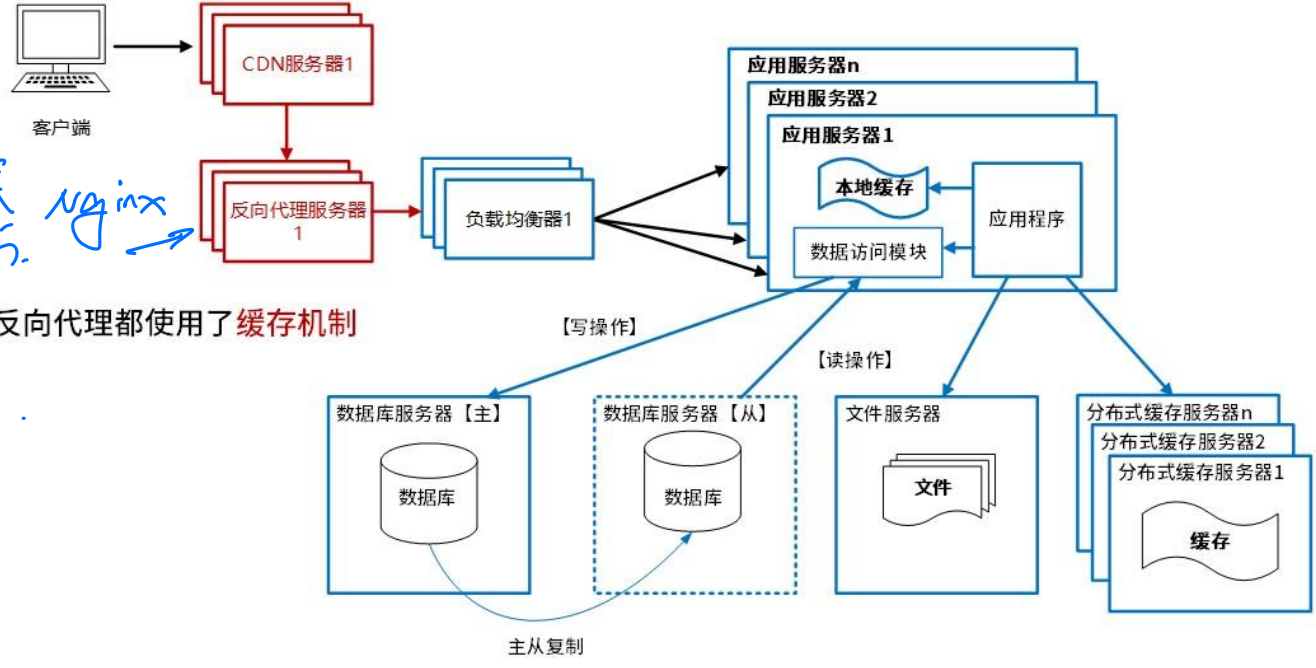
希赛





第六阶段：使用反向代理和CDN加速网站响应

希赛



CDN与反向代理都使用了缓存机制



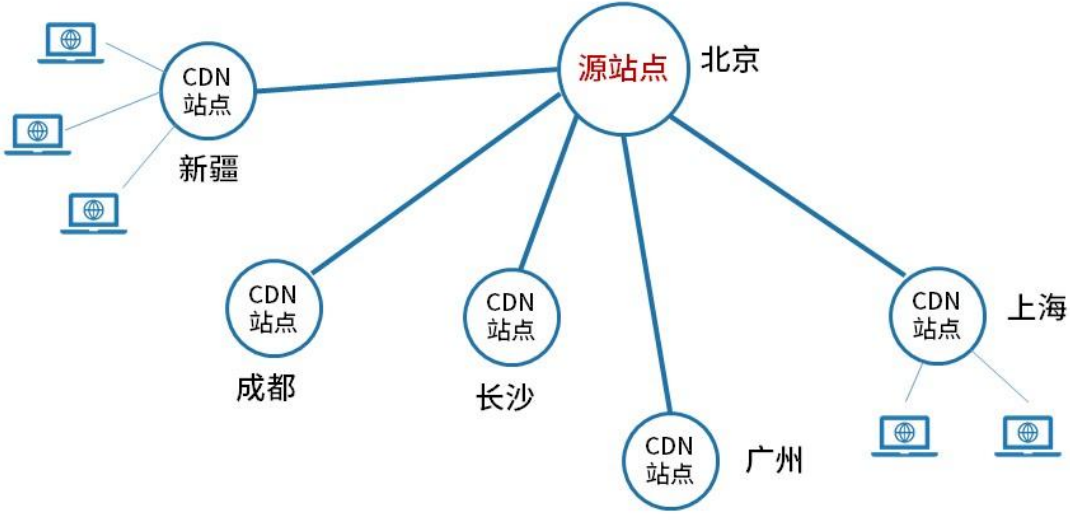
CDN（内容分发网络）

希赛

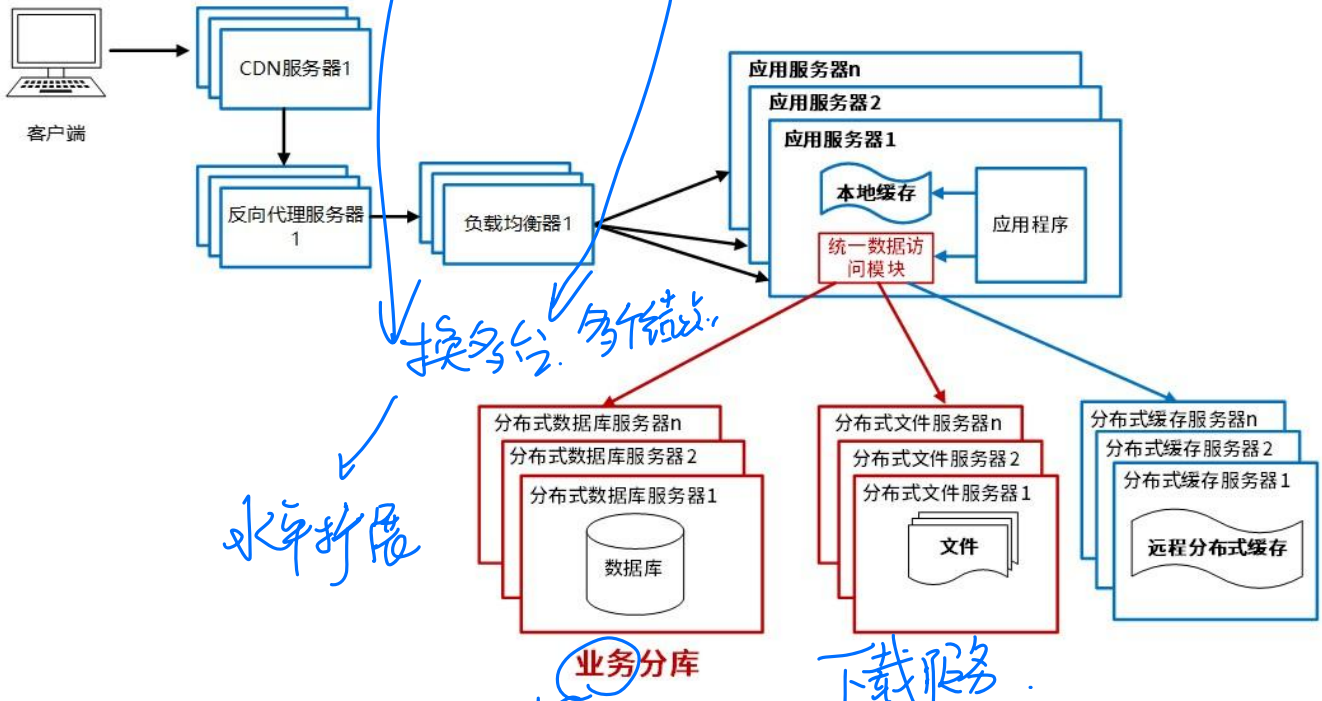
外网

CDN的全称是Content Delivery Network，即内容分发网络。其基本思路是尽可能避开互联网上有可能影响数据传输速度和稳定性的瓶颈和环节，使内容传输得更快、更稳定。

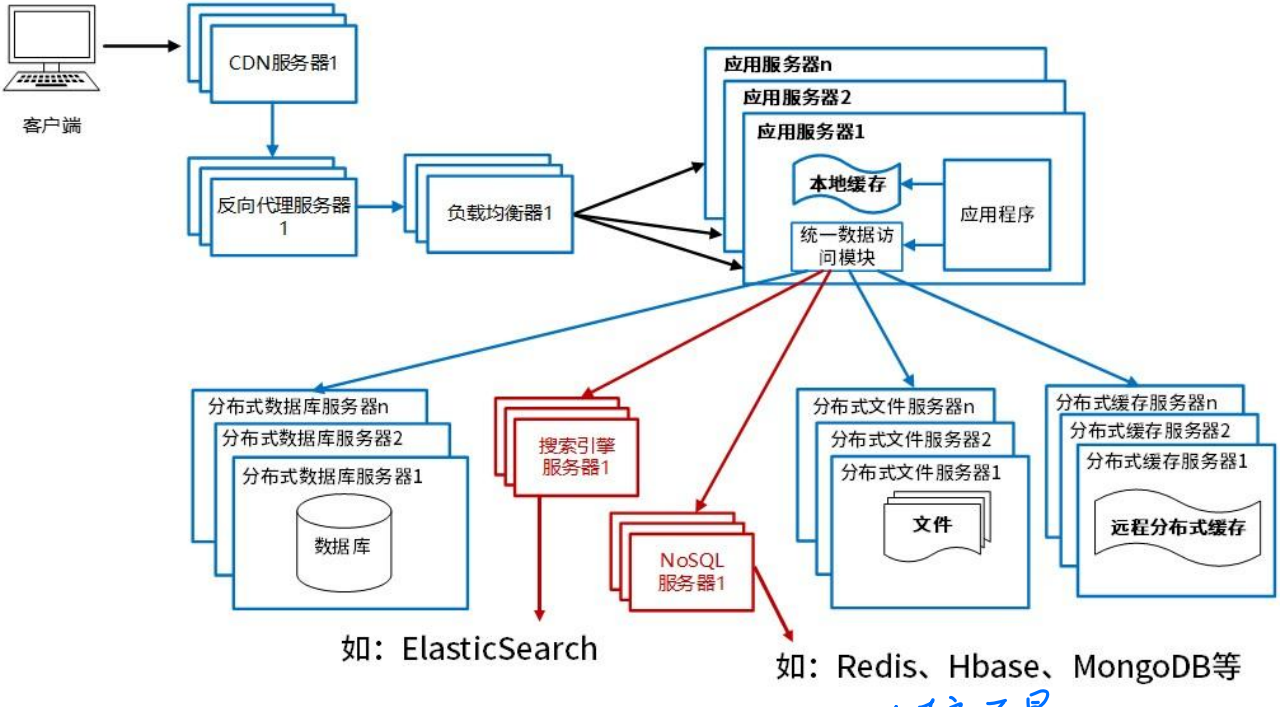
架构图



第七阶段：使用分布式文件系统和分布式数据库系统

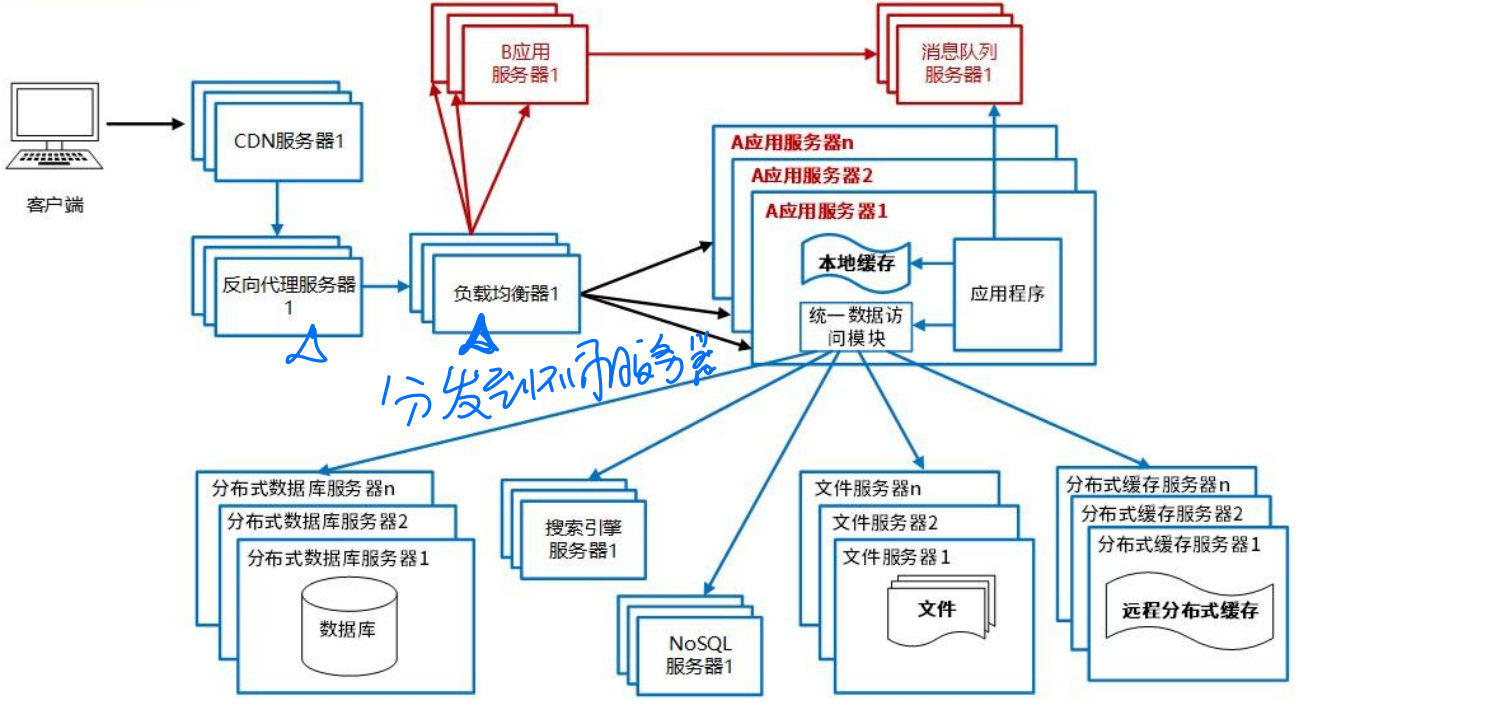


第八阶段：使用NoSQL和搜索引擎



第九阶段：业务拆分

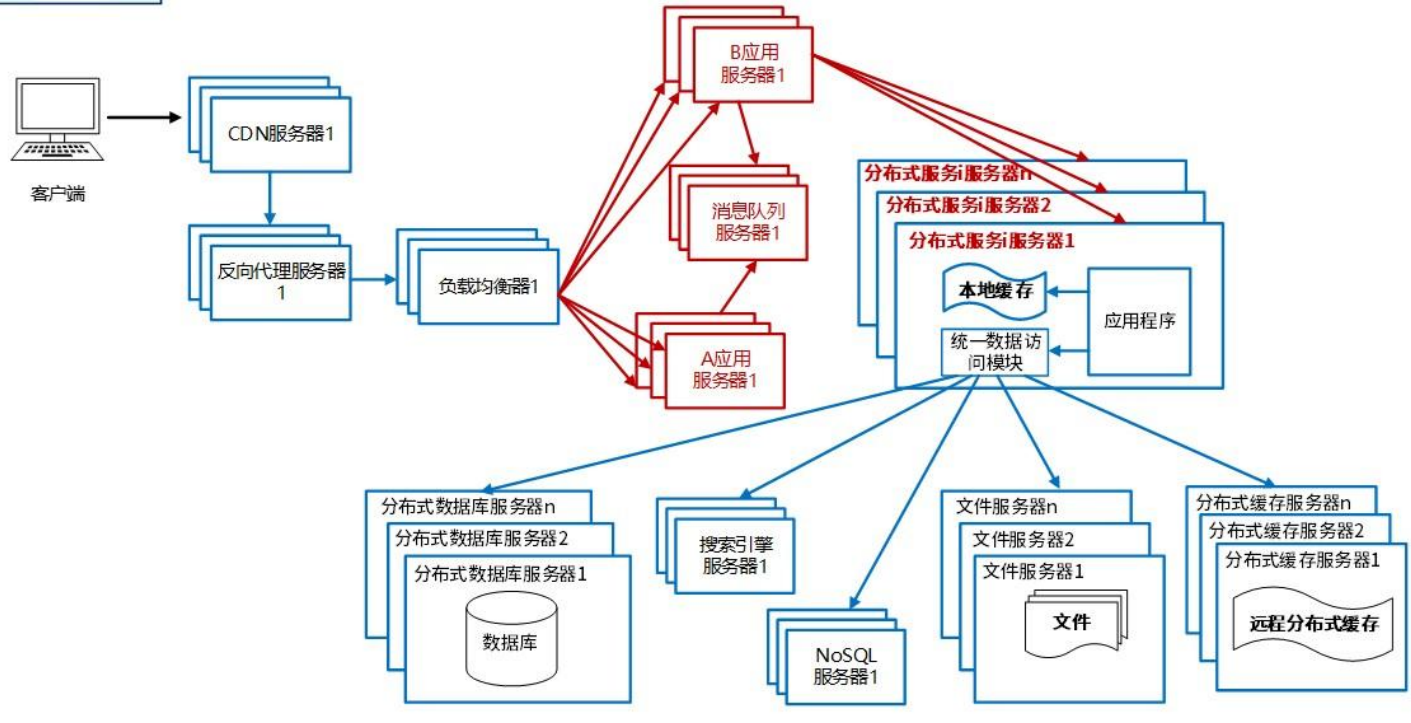



客户端
CDN服务器1
反向代理服务器1
负载均衡器1
B应用服务器1
消息队列服务器1
A应用服务器n
A应用服务器2
A应用服务器1
本地缓存
统一数据访问模块
应用程序
分布式数据库服务器n
分布式数据库服务器2
分布式数据库服务器1
数据库
搜索引擎服务器1
NoSQL服务器1
文件服务器n
文件服务器2
文件服务器1
文件
分布式缓存服务器n
分布式缓存服务器2
分布式缓存服务器1
远程分布式缓存

第十阶段：分布式服务

应用服务

希赛





Web应用服务器

希赛

WEB应用服务器可以理解为两层意思：

（1）WEB服务器：其职能较为单一，就是把浏览器发过来的Request请求，返回Html页面。

（2）应用服务器：进行业务逻辑的处理。如：EJB、Corba、COM+。

Apache：Web服务器，市场占有率60%左右。它可以运行在几乎所有的Unix、Windows、Linux系统平台上。

IIS：早期Web服务器，目前小规模站点仍有应用。

Tomcat：开源、运行servlet和JSP Web应用软件的基于Java的Web应用软件容器。

JBOSS：JBOSS是基于J2EE的开放源代码的应用服务器。一般与Tomcat或Jetty绑定使用。

WebSphere：一种功能完善、开放的Web应用程序服务器，它是基于Java的应用环境，用于建立、部署和管理Internet和Intranet Web应用程序。

WebLogic：BEA WebLogic Server是一种多功能、基于标准的web应用服务器，为企业构建自己的应用提供了坚实的基础。

Jetty：Jetty是一个开源的servlet容器，它为基于Java的web容器。



JWT

希赛

JWT（JSON Web Token）是一种用于身份验证和授权的开放标准。
它是一种轻量级的、基于JSON的令牌，可以在客户端和服务器之间传递信息。

JWT由三部分组成：
【头部】包含令牌类型和所使用的算法
【载荷】包含用户信息和其他元数据
【签名】用于验证令牌的完整性和真实性

JWT的应用场景：【信息交换】【授权】

不需要服务器端存储状态，安全地传递【非敏感信息】



JWT

希赛

【头部】 {
"alg": "HS256",
"typ": "JWT"
}

【载荷】 {
"sub": "1234567890",
"name": "xisaiwang",
"iat": 1516239022
}

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Inhpc2Fpd2FuZyIsImlhdCI6MTUxNjIzOTAyMn0.Jy5HGj1pIJs_OSc20at5ncMR0elabUUpHVVZN-6cenU

【签名】



响应式Web设计

希赛

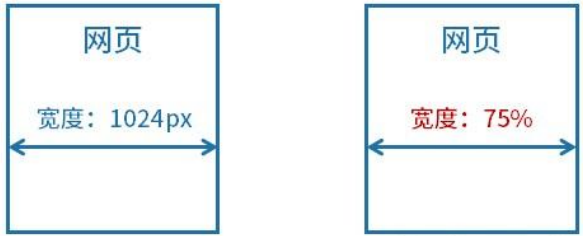
响应式WEB设计是一种网络页面设计布局，其理念是：集中创建页面的图片排版大小，可以智能地根据用户行为以及使用的设备环境进行相对应的布局。



方法与策略

（1）采用流式布局和弹性化设计：使用相对单位，设定百分比而非具体值的方式设置页面元素的大小。

（2）响应式图片：不仅要同比的缩放图片，还要在小设备上降低图片自身的分辨率。



响应式Web设计



关于中台

希赛

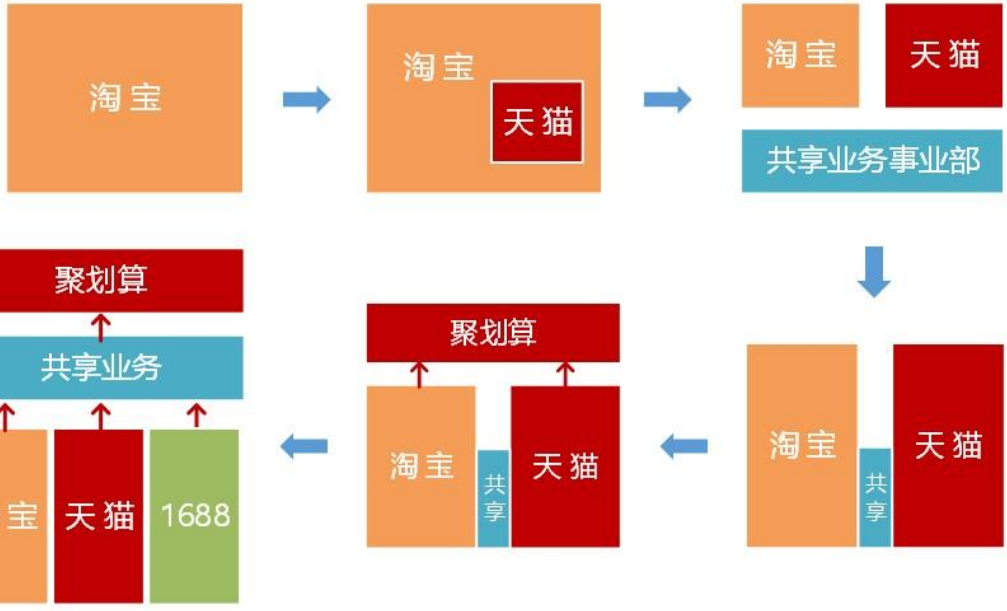
中台是一套结合互联网技术和行业特性，将企业核心能力以共享服务形式沉淀，形成“大中台、小前台”的组织和业务机制，供企业快速低成本的进行业务创新的企业架构。中台又可以进一步细分，比如业务中台、数据中台、XX中台。本质上，都是对企业通用进行业务创新的企业架构能力在不同层面的沉淀，并对外能力开放。

中台的践行者：

Supercell：芬兰移动游戏巨头，2015年世界游戏前10占5席，员工仅200多人，因使用中台，具有小团队快速开发能力，后被腾讯86亿美金收购。

阿里：2015年参观Supercell，而后推行中台。

关于中台





关于中台

希赛

淘宝 | 天猫 | 1688 | 聚划算 | .....

共享业务（中台）

用户中心 | 商品中心 | 交易中心

阿里云平台



关于中台

希赛

【业务中台】：提供重用服务，例如学员中心、课程中心之类的开箱即用可重用能力。

【数据中台】：提供数据整合分析能力，帮助企业从数据中学习改进，调整方向。

【技术中台】：提供技术重用组件能力，帮助解决基础技术平台的复用。如：中间件、分布式存储、AI、负载均衡等基础设施。

【业务中台】VS【数据中台】

- ✓ 多个电商渠道使用一个下单服务，一个订单接口同时为多个前台系统提供服务。
- ✓ 多个前台系统，根据一个用户的手机号，获取对应的画像、用户的标签。
- ✓ 将多个支付通道，抽象建立成一个支付API，暴露给前台业务系统。
- ✓ 通过一个订单编号，来获取可能的商品推荐清单，从而做到交叉销售。



关于中台

希赛

数据中台必备的4个核心能力

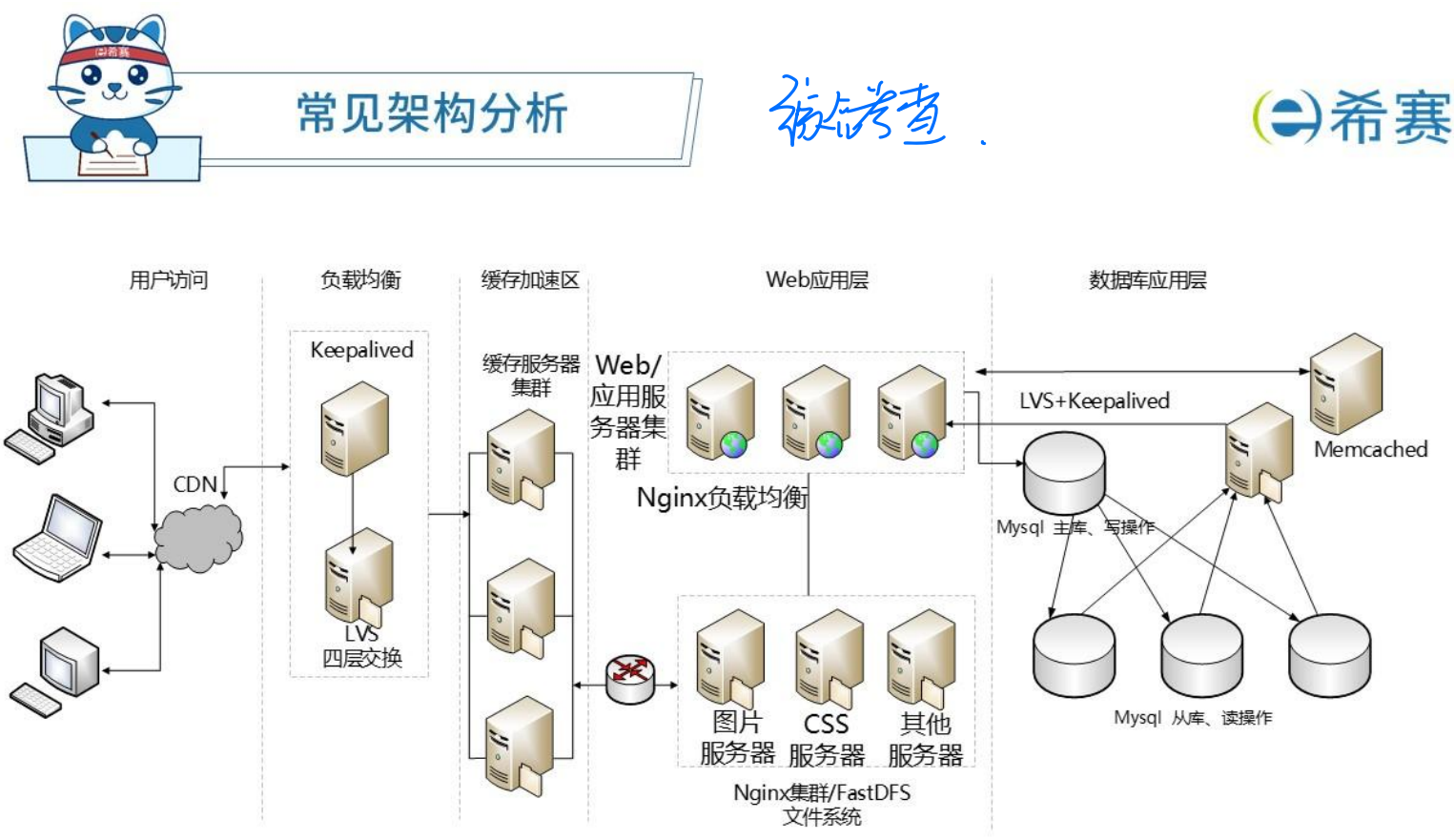
1、数据汇聚整合能力

2、数据提纯加工能力

3、数据服务可视化

4、价值变现方面

常见架构分析


用户访问
负载均衡
缓存加速区
Web应用层
数据库应用层
CDN
Keepalived
LVS
四层交换
缓存服务器
集群
Web/
应用服
务器集
群
Nginx负载均衡
LVS+Keepalived
Memcached
Mysql 主库、写操作
Mysql 从库、读操作
图片
服务器
CSS
服务器
其他
服务器
Nginx集群/FastDFS
文件系统